UNIVERSITÉ DE FIANARANTSOA

ÉCOLE NATIONALE D'INFORMATIQUE

MEMOIRE DE FIN D'ETUDES POUR L'OBTENTION DU DIPLÔME DE MASTER

Mention : Intelligence Artificielle

Parcours : Gouvernance et Ingénierie de Données

Intitulé : Conception, réalisation et intégration d'une intelligence artificielle privée dans Odoo 17

Présenté le 3 decembre 2026
Par Monsieur RAILALA Mahalahatse
Membres du Jury :

Président : Monsieur RALAIVAO Jean Christian, Maître de Conférences

Examinatrice : Madame RATIANANTITRA Volatiana Marielle, Maître de Conférences

Rapporteurs :

- Madame RAKOTOASIMBAHIAKA Antsa Cyprienna
- Monsieur Tahinjanahary Maminirina , Ingénieur en Inf

Année Universitaire 2025-202

miro Knowledge Engineering
Software System Solution Provider
Lot H 73 bis Ambohidratrimo
105 - Antananarivo Madagascar
Tél. : 032 07 174 74
E-mail : contact@miroke.com

CANEVAS

Curriculum Vitae.....I

Sommaire II

Remerciements.....III

Liste des figures.....IV

Liste des tableaux V

Liste des abréviations (ou Nomenclature) VI

Introduction générale.....1 (début chiffre arabe)

PARTIE I. PRESENTATIONS .....

Chapitre 1. Présentation de l'ENI .....

Chapitre 2. Présentation de l'établissement d'accueil (société, entreprise, etc).....

Chapitre 3. Description du projet.....

3.1 Formulation.....

3.2.Objectif et besoins de l'utilisateur.....

3.3.Moyens nécessaires à la réalisation du projet (Humain, Matériel, Logiciel) .....

3.4. Résultats attendus.....

3.5. Chronogramme de travail.....

PARTIE II. ANALYSE ET CONCEPTION.....

Chapitre 4. Analyse préalable.....

4.1. Analyse de l’existant .....

....

4.1.1. Organisation actuelle (traitement actuel et personnel impliqué).....

4.1.2. Inventaire des moyens matériels et logiciels.....

4.2. Critique de l’existant (points forts et points faibles).....

4.3. Conception avant-projet .....

4.3.1. Proposition des solutions .....

4.3.2. Méthodes de conception et outils utilisés (Choix, Justification et présentation) .....

(Attendu : montrer clairement le problème que l’application va résoudre, en se basant sur les limites des outils ou

systèmes existants)

Chapitre 5. Analyse conceptuelle.....

5.1.Désignation des rôles de l’équipe SCRUM.....

5.2.Etapes de l’élaboration du product backlog .....

5.3.Product backlog .....

.....

5.4. Sprint backlog .....

.....

5.4.1. Présentation du travail à faire (tableau release/sprint) .....

Canevas Pour IA Dev

5.4.2. Planification du sprint (tableau sprint/ tâches).....

5.5. Dictionnaire des données .....

5.6. Règles de gestion .....
.....

5.7.Diagramme de cas d’utilisation global.....

5.8. Modèle du domaine.....
....

5.9. Diagramme de classe de conception pour chaque sprint .....

5.10. Diagramme de classe de conception globale .....

5.9. Diagramme de paquetage .....

5.10. Diagramme de déploiement .....

Chapitre 6. Intégration IA dans la conception ....

6.1. Définition du module IA (l’IA que vous allez traiter).....

6.2. Données nécessaires (les données nécessaires à votre outil/modèle).....

6.3. Choix du modèle IA et justification (comparaisons des outils ou modèles utilisés) .....

6.4. Méthode d'intégration (API REST, microservice, modèle embarqué) .....

(Attendu : décrire précisément où et comment l'IA interagit avec le reste du système (par ex. : prédiction dans le

backend, suggestion côté front, moteur de recommandation branché sur la BDD))

PARTIE III. REALISATION.....

Chapitre 7. Mise en place de l'environnement de développement .....

7.1.Installation et configuration des outils .....

7.2. Architecture de l'application.....

7.3. Déploiement des environnements (dev, test, prod) .....

(Attendu : prouver que l'étudiant sait installer et configurer correctement les outils nécessaires (Ex : Spring Boot,

Django, React, Angular, MySQL, etc.).

Chapitre 8. Développement de l'application .....

8.1.Création et structuration de la base de données.....

8.2.Développement du frontend (interfaces utilisateur) .....

8.3.Développement du backend (API, logique métier) .....

8.4.Développement et entraînement du module IA .....

8.5.Intégration IA–
application ....

(Attendu : présenter le code principal (captures, extraits commentés), expliquer comment les différentes couches

communiquent et montrer le résultat final (captures d’écran, tests réels))

Chapitre 9. Tests et
validation .....

9.1.Tests unitaires et
d’intégration.....

9.2.Validation
fonctionnelle .....
....

9.3.Évaluation de la performance du module
IA .....

(Attendu : prouver que l’application marche vraiment (tests utilisateurs, jeux de test, résultats mesurés). Les étudiants

doivent montrer qu’ils savent valider un projet logiciel comme un produit fini.)

Conclusion.....
.....(fin chiffre arabe)

Bibliographie (ou Références
bibliographiques)..... VII

Webographie (ou Références
webographiques).....VIII

Glossaire.....
.....IX

Annexes .....
.....X

Table des
matières.....
.....XI

Résumé .....
.....

Abstract.....
.....

Quelques règlements à respecter dans le manuscrit

1) Bien justifier l'alignement des textes (gauche, droite)
2) Utiliser Times New Roman 12
3) Ne pas souligner les titres, mettre en gras les titres
4) Uniformiser les puces utilisés dans tous le document
5) Format CV simple et clair (et non pas le style de CV pour postuler)
6) Le sommaire propose un résumé des parties, chapitres et les titres au premier degré (Exemple : le sommaire
se termine au 1.1) alors que la table des matières correspond à une liste complète des parties, chapitres, et les
titres à n degré de tous les documents (Exemple :1.1.1)
7) Dans les Remerciements
- Citation par tiret
- Utilisation de je, nous autorisé dans le remerciement mais pour le reste du document NON
- Ordre de remerciement : Président de l'Université, Directeur de l'Ecole, Directeur de la société de stage,
Chef de mention, Chef de parcours, Les rapporteurs, Les autres membres du Jury, etc.
- Madame/Monsieur (Nom et Prénoms), titre, fonction, raison pour laquelle on remercie la personne
8) A trier par ordre alphabétique :
- Nomenclature
- Dictionnaire des données
- Glossaire
- Mots-clés et Keywords (minimum 5 mots, maximum 8 mots)
9) Actualiser la présentation de l'ENI
10) Les Légendes :
- Légende des figures en bas
- Légende des tableaux en haut
11) Il faut toujours mettre une phrase introductive présentant les figures et les tableaux
12) Ne pas utiliser : ci-dessus, ci-dessous, précédent, suivant, Voici, Voilà
13) Références :
- Présentation Bibliographie : [1] Auteur, titre de l'ouvrage, année d'édition (publication) , Edition
(éditeur, lieu), nombre de page
- Présentation Webographie : [2] url , titre de la page, date de consultation.
Exemple : [2] https:// ...., Titre du page, consulté le 18 octobre 2021
- Si la bibliographie se termine par [6], le webographie commence par [7]

CURRICULUM VITAE

ETAT CIVIL

Nom et prénom : RAILALA Mahalahatse
Adresse : Antananarivo, Madagascar
Tel : +261 32 44 552 52
E-mail : mahalahatserailala@gmail.com | LinkedIn

FORMATIONS et DIPLOME OBTENU

1. 2024 – 2026 : Master 2 en Intelligence Artificielle — Parcours Gouvernance et Ingénierie de Données — ENI, Université de Fianarantsoa
2. 2019 – 2025 : Certificat d'Aptitude Pédagogique de l'École Normale en Mathématiques — ENS, Université de Toliara
3. 2018 – 2019 : Baccalauréat Série C — Houssen Memorial School

STAGES ET EXPERIENCES PROFESSIONNELLES

1. **2023-2024**

   Développeur Odoo, Aout - Novembre 2024
   Stage au sein de l'entreprise MIRO KE ayant pour thème : « Conception, réalisation et intégration du module menu des applications et campagne d'e-mailing brevo dans Odoo 17 »**.**

   Développeur Javascript et python, Juillet 2024 :
   Projet au sein de l'ENI ayant pour thème « Conception et réalisation d'un réseau social » en utilisant React native et Django.
   Projet au sein de l'ENI ayant pour thème « Réalisation de gestion des étudiants de l'ENI » en utilisant React js, et Node js.
2. **2022-2023**

   Développeur C# et .NET, Aout - Novembre 2023 :
   Stage au sein du VFTM ayant pour thème : « Conception et réalisation d'une application de gestion de stock ».
   Développeur Backend, Juin - Juillet 2023 :
   Projet au sein de l'ENI ayant pour thème « Réalisation d'une application desktop de gestion de stock » en utilisant Java et « Réalisation d'une application web pour la gestion des examens » en utilisant Bootstrap 5 et PHP.
3. **2021-2022**

   Développeur Frontend, Octobre 2022 :
   Projet de fin d'année au sein de l'ENI ayant pour thème : « gestion de vol d'une compagnie aérienne. » en utilisant QT Creator et C++.

COMPÉTENCES EN INFORMATIQUE

- **Langages de programmation :** C, Python.
- **Langage de requête :** SQL.

- **Technologies web :** JavaScript, HTML.
- **Frameworks:** React js, Next js, Express js, OWL js.
- **Environnement de développement :** Visual Studio, Visual Studio Code, NetBeans, PyCharm, Eclipse.
- **Systèmes de gestion de base de données :** MySQL, PostgreSQL**.**
- **Notation et méthode de conception :** UML, 2TUP, MERISE.
- **Systèmes d'exploitation :** Windows (7,10,11), Ubuntu 22, Debian 11.
- **Outils de conception** : Visual Paradigm, Win 'Design
    - **Outils de Bureautique** : Word, PowerPoint, Excel, Access
- **IA & ML :** RAG, LLM, LangChain, QLoRA, CatBoost, XGBoost
- **MLOps :** MLflow, DVC, GitHub Actions, Evidently AI, Docker, Git
- **Données :** PostgreSQL, MongoDB, Qdrant, Talend, Power BI, Knime, Alteryx
- **Langages :** Python, Java, JavaScript, Flutter

CONNAISSANCES LINGUISTIQUES

Langues	Comprendre à l'audition					Lire						Écrire et Rédiger						Parler et Communiquer					
	TB	B	AB	P	NS	TB	B	AB	P	NS		TB	B	AB	P	NS		TB	B	AB	P	NS	
Français	X					X						X						X					
Anglais	X					X						X						X					

Grille d'évaluation : TB : Très Bien, B : Bien, AB : Assez Bien, P : Passable, NS : Niveau scolaire

SOMMAIRE

CURRICULUM VITAE.....I
SOMMAIRE.....III
REMERCIEMENTS.....V
LISTE DES FIGURES.....VI
LISTE DES TABLEAUX.....IX
LISTE DES ABREVIATIONS.....X
INTRODUCTION GENERALE.....1
PARTIE I : PRESENTATIONS.....2
Chapitre 1. Présentation de l'ENI.....3
1.1. Information d'ordre général.....3
1.2. Missions et historique.....3
1.3. Organigramme institutionnel de l'ENI.....5
1.4. Domaine de spécialisation.....6
1.5. Architecture des formations pédagogiques.....6
1.6. Relations de l'ENI avec les entreprises et les organismes.....9
1.7. Partenariat au niveau international.....10
1.8. Débouchés professionnels et diplômés.....11
1.9. Ressources humaines.....13
Chapitre 2. Présentation de l'établissement d'accueil.....14
2.1. Information de l'entreprise.....14
2.2. Organisation du MIRO KE.....14
2.3. Missions.....14
2.4. Partenariat.....15
2.5. Valeurs fondamentales.....15
Chapitre 3. Description du projet.....17
3.1. Formulation.....17
3.2. Objectif et besoins de l'utilisateur.....17
3.3. Moyens nécessaires à la réalisation du projet.....18
3.4. Résultats attendus.....20

3.5. Chronogramme de travail.....20
PARTIE II: ANALYSE ET CONCEPTION.....21
Chapitre 4. Analyse préalable.....22
4.1. Analyse de l'existant.....22
4.2. Critique de l'existant.....23
4.3. Conception avant-projet.....24
Chapitre 5. Analyse conceptuelle.....32
5.1. Présentation de la méthode 2TUP.....32
5.2. Dictionnaire des données.....33
5.3. Règles de gestion.....35
5.4. Représentation et spécification des besoins.....36
Chapitre 6. Conception détaillée.....55
6.1. Architecture système.....55
6.2. Diagramme de séquence de conception pour chaque cas d'utilisation.....56
6.3. Diagramme de classe de conception pour chaque cas d'utilisation.....63
6.4. Diagramme de classe de conception global.....68
6.5. Diagramme de paquetage.....68
6.6. Diagramme de déploiement.....69
PARTIE III : REALISATION.....70
Chapitre 7. Mise en place de l'environnement de développement.....71
7.1. Installation et configuration des outils.....71
7.2. Architecture de l'application.....74
Chapitre 8. Développement de l'application.....76
8.1. Création de la base de données.....76
8.2. Codage de l'application.....77
8.3. Présentation de l'application.....80
CONCLUSION.....87
WEBOGRAPHIE.....X
GLOSSAIRE.....X
ANNEXE.....XIII
TABLE DES MATIERES.....XVI

REMERCIEMENTS

Le présent rapport n'aurait jamais pu être réalisé sans la contribution et la participation de diverses personnes de bonne volonté. Nous tenons également à exprimer nos sincères remerciements à toutes celles et ceux qui ont soutenu dans la réalisation de ce mémoire.
Nos remerciements s'adressent en particulier à :

- Monsieur HAJALALAINA Aimé Richard, Président de l'université de Fianarantsoa, grâce à lui nous avons pu poursuivre nos études dans ce cursus.
- Monsieur MAHATODY Thomas, Directeur de l'Ecole Nationale d'Informatique pour les connaissances acquises durant la formation, ayant grandement aidé à ces projets.
- Monsieur RAKOTONDRATSIMA Christian, Directeur de l'entreprise MIRO K.E. pour avoir accueilli, facilité l'intégration et offert une expérience professionnelle enrichissante.
- Monsieur DIMBISOA WIlliam, Maître de Conférences et Responsable de la Mention Intelligence Artificielle à l'Ecole Nationale d'Informatique. Ses orientations ont joué un rôle clé dans la réussite de cette formation et dans la réalisation de ce mémoire.
- Madame RATIANANTITRA Volatiana Marielle, Maître de conférences et Responsable du parcours Gouvernance et Ingénierie de Données d'avoir examiné la présentation de ce mémoire.
- Monsieur RATIARSON Venot, Maître de Conférences étant donné son désir d'être notre encadreur pédagogique.
- Monsieur MANDIMBINIRINA José, Ingénieur en informatique son rôle en tant qu'encadreur professionnel, qui a largement facilité notre intégration et enrichi notre expérience.
- Tous les enseignants et à toutes les enseignantes de l'Ecole Nationale d'Informatique pour leur précieuse assistance pédagogique et les travaux pratiques.
- Toutes les équipes du MIRO K.E pour leurs aimables collaborations.
- Ma famille surtout mes parents et mes frères pour leur soutien moral, physique, financier et matériel dans la réalisation de ce mémoire. Sans eux, il serait difficile de poursuivre mes rêves.

LISTE DES FIGURES

LISTE DES TABLEAUX

Tableau 1.Architecture des études correspondant au système LMD 6
Tableau 2.Mention et parcours au sein de l'ENI 6
Tableau 3.Liste des formations existantes à l'ENI 8
Tableau 4.Débouchés éventuels des jeunes diplômés 12
Tableau 5.Information de l'entreprise MIRO KE 14
Tableau 6.Moyens Humains 18
Tableau 7.Moyens matériels 19
Tableau 8.Moyens logiciels 19
Tableau 9.Matériels au sein du MIRO KE 22
Tableau 10.Analyse comparative de solutions proposées 24
Tableau 11.Analyse comparative de UP et MERISE 25
Tableau 12.Analyse comparative du langage python et Java 27
Tableau 13.Analyse comparative du framework OWL js et React js 28
Tableau 14.Analyse comparative de PostgreSQL et MySQL 29
Tableau 15.Analyse comparative de ERPNext et Odoo 30
Tableau 16.Dictionnaire des données 33
Tableau 17.Règles de gestion 36
Tableau 18.Identification de cas d'utilisation 37
Tableau 19.Priorisation de cas d'utilisation 45

LISTE DES ABREVIATIONS

- 2TUP : Two Track Unified Process
- A : Alphabétique
- AN : Alpha Numérique
- ASR : Administration des Systèmes et Réseaux
- API : Application Programming Interface
- B : Booléen
- BD : Base de Données
- BTS : Brevet de Technicien Supérieur
- CA : Conseil d'Administration
- CPU : Central Processing Unit
- CNH : Commission Nationale d'Habilitation
- CRM : Customer Relationship Management
- CSS : Cascading Style Sheet
- CTO : Chief Technology Officer
- CU : Cas d'utilisation
- CUR : Centre Universitaire Régionale
- CRM : Customer Relationship Management
- D : Date
- DEA : Diplôme d'Etudes Approfondies
- DUT : Diplôme Universitaire de Technicien
- ENI : Ecole Nationale d'Informatique
- ERP : Enterprise Ressource Planning
- GANT : Graphique d'Allocation de Nombre Total
- GID : Gouvernance et Ingénierie de Données
- HDD : Hard Disk Drive
- HDR : Habilitation à Diriger des Recherches
- HTML : HyperText Markup Language
- HTTPS : HyperText Transfer Protocol Secure

- IA : Intelligence Artificielle
- IDE : Integrated Development Environment
- IG : Informatique Générale
- LMD : Licence-Master-Doctorat
- MIRO K.E.: Miro Knowledge Engineering
- MRP : Manufacturing Resource Planning
- MVC : Model View Controller
- N : Numérique
- OCC : Objets connectés et Cybersécurité
- ORM : Object Relational Mapping
- OWL : Odoo Web Library
- OS : Operating System
- PRESUP : Programme de renforcement en l'Enseignement Supérieur
- RAM : Random Access Memory
- RG : Règles de gestion
- SGBD : Système de Gestion de Base de Données
- SGBDR : Système de Gestion de Base de Données Relationnel
- SSD : Solid State Drive
- SQL : Structured Query Language
- TIC : Technologie de l’Information et de la Communication
- UI/UX : User Interface/User Experience
- UML : Unified Modeling Language
- VSCode : Visual Studio Code
- XML : eXtensible Markup Language

INTRODUCTION GENERALE

À l'ère de la transformation numérique, les systèmes d'information d'entreprise ont atteint un niveau de maturité où l'optimisation des flux opérationnels repose sur l'intégration continue et l'interopérabilité des données. Au cœur de cette architecture, les Progiciels de Gestion Intégrés (PGI / ERP), à l'instar d'Odoo, fédèrent l'ensemble des processus critiques de l'organisation : gestion de la chaîne logistique, comptabilité et finances, relations clients (CRM), ressources humaines et production industrielle. Parallèlement, l'avènement spectaculaire des modèles de fondation et de l'intelligence artificielle générative (LLM — Large Language Models) redéfinit en profondeur les paradigmes d'interaction homme-machine, offrant la perspective inédite d'interroger, d'analyser et d'orchestrer ces gisements de données complexes au travers d'un dialogue naturel intuitif.

Cependant, l'adoption effective de l'intelligence artificielle au sein des ERP se heurte à des contraintes de sécurité et de gouvernance strictes. L'utilisation d'API d'IA générative en mode SaaS (hébergées sur des clouds publics tiers) pose des risques majeurs et inacceptables de fuite de données hautement sensibles, d'exposition du secret des affaires et de non-conformité vis-à-vis des cadres légaux relatifs à la protection des données à caractère personnel (tels que la loi n° 2014-038 à Madagascar et le RGPD en Europe). Pour les organisations, la dépendance envers des fournisseurs externes induit également des coûts récurrents indexés sur le volume de jetons (tokens) et une absence totale de maîtrise sur le cycle de vie, la traçabilité et la localisation des données traitées. L'impératif stratégique consiste dès lors à concevoir une solution d'intelligence artificielle « privée » et auto-hébergée (on-premise), opérant en circuit fermé sur l'infrastructure souveraine de l'entreprise.

Au-delà de la souveraineté des modèles, l'intégration d'un agent conversationnel au sein d'un ERP tel qu'Odoo 17 soulève trois défis d'ingénierie majeurs :

- **La sécurité d'accès et le respect strict du contrôle d'accès basé sur les rôles (RBAC) :** L'accès aux enregistrements transactionnels (devis, factures, fiches employés) ne peut souffrir d'aucune élévation implicite de privilèges. Chaque requête de l'IA doit être exécutée strictement sous l'identité de l'utilisateur demandeur (uid) et dans son contexte de cloisonnement multi-sociétés (allowed_company_ids), en respectant à la lettre les listes de contrôle d'accès (ir.model.access) et les règles d'enregistrement dynamiques (ir.rule). Tout contournement par des droits administrateur transversaux (SUPERUSER_ID ou appel à sudo()) est formellement proscrit.
- **La prévention des hallucinations et l'exactitude factuelle :** Les modèles de langage sont naturellement sujets aux affabulations sémantiques. Dans un contexte de gestion d'entreprise où une erreur de chiffre ou de statut a des répercussions opérationnelles graves, l'agent doit reposer sur des mécanismes rigoureux de récupération de données vérifiables (RAG et requêtes ORM structurées) accompagnées d'une citation formelle des sources.
- **L'intégration ergonomique native et la traçabilité dans l'ERP :** L'interface conversationnelle doit faire corps avec l'environnement de travail d'Odoo sans perturber les

usages quotidiens, tout en assurant une journalisation complète et auditable des interactions pour répondre aux impératifs de conformité.

Afin de répondre à ces enjeux cruciaux, l'entreprise MIRO K.E. (Silver Partner Odoo) nous a confié la réalisation d'un projet de fin d'études axé sur la conception, la réalisation et l'intégration d'une intelligence artificielle privée et souveraine dans Odoo 17. Dans le cadre de ce mémoire de Master, nous délimitons et validons expérimentalement le cœur opérationnel de ce système, articulé autour de deux composantes logicielles fondamentales :

- **Le module addon Odoo 17 (« miro_ai ») :** conçu nativement avec le framework JavaScript réactif OWL (Odoo Web Library), ce module offre une interface conversationnelle ergonomique intégrée à la barre supérieure (systray) et au chatter d'Odoo. Il intègre un contrôleur sécurisé avec signature cryptographique HMAC, relais du contexte de session utilisateur (uid, company_id, groupes), gestion asynchrone des requêtes par scrutation de tâches (job polling) et journalisation d'audit (miro.ai.log).
- **Le backend d'orchestration et d'intelligence artificielle privée :** déployé sous FastAPI et orchestré via LangGraph, il couple un moteur RAG hybride performant (BGE-M3 dense + Rank-BM25 avec fusion Reciprocal Rank Fusion) pour l'interrogation documentaire et un agent de consultation temps réel de l'ORM Odoo sous privilèges stricts sans aucune élévation de privilèges.

Les dimensions avancées que constituent l'interconnexion universelle par serveur MCP (Model Context Protocol) et l'intelligence de code (Code Intelligence par graphe sémantique) sont quant à elles positionnées et détaillées comme des perspectives structurantes majeures du projet.

Le présent mémoire s'articule autour de trois parties fondamentales :

- **La Partie I (Présentations)** introduit les contextes académique et professionnel en présentant successivement l'École Nationale d'Informatique (ENI) et l'entreprise d'accueil MIRO K.E., avant d'exposer en détail la description du projet, ses objectifs, ses moyens et son chronogramme prévisionnel.
- **La Partie II (Analyse et Conception)** procède à l'analyse préalable de l'existant au sein de MIRO K.E., puis détaille la conception architecturale globale, la modélisation des cas d'utilisation, les flux de sécurité et les choix méthodologiques et technologiques retenus.
- **La Partie III (Réalisation)** documente l'environnement d'implémentation, l'intégration du module miro_ai au sein de l'écosystème Odoo 17, ainsi que les protocoles de tests, les benchmarks de performance et l'analyse critique des résultats obtenus.

PARTIE I : PRESENTATIONS

Présentation de l'Ecole Nationale d'Informatique

L'École Nationale d'Informatique (ENI) est reconnue comme un établissement de référence à Madagascar dans le domaine de la formation en informatique.

1. Information d'ordre générale

L'Ecole Nationale d'Informatique, en abrégé ENI, est un établissement d'enseignement supérieur rattaché académiquement et administrativement à l'Université de Fianarantsoa. Le siège de l'Ecole se trouve à Tanambao-Antaninarenina à Fianarantsoa. L'adresse pour la prise de contact avec l'Ecole est la suivante : Ecole Nationale d'Informatique (ENI) Tanambao, Fianarantsoa. Le numéro de sa boîte postale est 1487 avec le code postal 301. Téléphone : 038 96 566 96 ou 034 76 591 95. Son adresse électronique est la suivante : **eni@eni.mg**. Il dispose également d'un site web : **www.eni.mg**

2. Missions et historiques

L'ENI se positionne sur l'échiquier socio-éducatif malgache comme étant le plus puissant secteur de diffusion et de vulgarisation des connaissances et des technologies informatiques.

Cette Ecole Supérieure peut être considérée aujourd'hui comme la vitrine et la pépinière des élites informaticiennes du pays.

De façon formelle, l'ENI était créée par le décret N° 83- 185 du 24 Mai 1983, comme étant le seul établissement Universitaire Professionnalisé au niveau national, destiné à former des techniciens et des Ingénieurs de haut niveau, aptes à répondre aux besoins et exigences d'Informatisation des entreprises, des sociétés et des organes implantés à Madagascar.

L'ENI a pour conséquent pour mission de former des spécialistes informaticiens compétents et opérationnels de différents niveaux notamment :

- En fournissant à des étudiants des connaissances de base en informatique ;
- En leur transmettant le savoir-faire requis, à travers la professionnalisation des formations dispensées et en essayant une meilleure adéquation des formations par rapport aux besoins évolutifs des sociétés et des entreprises ;
- En initiant les étudiants aux activités de recherche dans les différents domaines des Technologies de l'Information et de la Communication (TIC) ;

La filière de formation d'Analystes Programmeurs a été mise en place à l'Ecole en 1983, et a été gelée par la suite en 1996, tandis que la filière de formation d'ingénieurs a été ouverte à l'Ecole en 1986.

Une formation de troisième cycle a été ouverte à l'Ecole a été ouverte à l'Ecole depuis l'année 2003 – 2004 grâce à la coopération académique et scientifique entre l'Université de Fianarantsoa pour le compte de l'ENI et l'Université Paul Sabatier de Toulouse (UPST).

Cette filière avait pour objectif de former certains étudiants à la recherche dans les différents domaines de l'Informatique, et notamment pour préparer la relève des Enseignants-Chercheurs qui étaient en poste.

Pendant l'année 2007-2008, la formation en vue de l'obtention du diplôme de Licence Professionnelle en Informatique a été mise en place à l'ENI avec les deux parcours de formation:

- Génie Logiciel et base de Données.
- Administration des Système et réseaux.

La mise en place à l'Ecole de ces deux options de formation devait répondre au besoin de basculement vers le système Licence – Master – Doctorat (LMD).

En vue de surmonter les difficultés de limitation de l'effectif des étudiants accueillis à l'Ecole, notamment à cause du manque d'infrastructures, un système de « Formation Hybride » a été mise en place à partir de l'année 2010. Il s'agit en effet d'un système de formation semi présentielle et à distance avec l'utilisation de la visioconférence pour la formation à distance. Le système de formation hybride a été ainsi créé à Fianarantsoa ainsi qu'Université de Toliara. Cette formation est à l'origine du parcours Informatique Générale.

En 2023, la mention Intelligence Artificielle (IA) a été ouvert au sein de l'Ecole pour répondre les besoins des entreprises. La formation est destinée aux étudiants titulaires du diplôme de licence (Bac +3) en Mathématiques ou en Statistiques ou en Informatique, etc.

La mention IA comporte deux parcours :

- Gouvernance et Ingénierie de Données (GID),
- Objets connectés et Cybersécurité (OCC).

Le principe de l'enseignement pour le parcours GID offre aux l'étudiants des compétences scientifiques et techniques spécialisées en Science de données. Pour le parcours OCC, les étudiants octroient la double spécialité premièrement en internet des objets et deuxièmement en

cybersécurité. La formation de master est axée sur l'ensemble d'applications de l'Intelligence Artificielle.

Depuis l'année universitaire 2024-2025, la mention Expertise Digitale est officiellement ouverte au grand public. Cette mention propose deux (2) parcours distincts :

- Métiers du Digital (MDi) – disponible aux niveaux Licence et Master. Ce parcours est destiné aux étudiants titulaires du diplôme de Baccalauréat, toutes séries confondues. Il est élaboré pour équiper les étudiants des compétences essentielles dans les domaines du marketing digital, de la communication digitale, et de la gestion de projets digitaux.
- Audit des Systèmes d'Information (ASI) – proposé au niveau Master uniquement. Ce parcours s'adresse aux étudiants titulaires d'une licence (Bac +3) en Mathématiques, en Statistiques ou en Informatique. Il est structuré pour approfondir les connaissances théoriques et pratiques nécessaires à l'audit efficace des systèmes d'information.

Le système d'enseignement adopté pour cette mention est un système en ligne. Ainsi, les cours et ressources pédagogiques sont accessibles en permanence via la plateforme numérique Moodle. Par ailleurs, des regroupements en présentiel ou en ligne seront organisés périodiquement, conformément au planning défini par les responsables pédagogiques.

À partir de l'année universitaire 2026, dans le cadre de la réorganisation de l'offre de formation, les anciens parcours de la Mention Informatique ont été restructurés et renommés afin de mieux refléter l'évolution des domaines et des compétences visés. Ainsi, l'ancien parcours Génie Logiciel et Base de Données (GB) devient le parcours Ingénierie Logicielle et Systèmes d'Information (ILSI), le parcours Administration des Systèmes et Réseaux (ASR) devient Ingénierie des Systèmes et Réseaux Informatiques (ISRI), tandis que le parcours Informatique Générale (IG) devient Ingénierie des Systèmes d'Information Intégrés (ISII). Cette évolution vise notamment à actualiser les intitulés et à renforcer la cohérence entre les formations proposées et les besoins actuels du secteur numérique.

3. Organigramme institutionnel

L'organigramme de l'Ecole est inspiré des dispositions du décret N° 83-185 du 24 Mai 1983. L'ENI est administrée par un Conseil d'Ecole, et dirigée par un directeur nommé par un décret adopté en Conseil des Ministres. Le Collège des enseignants regroupant tous les

enseignants-chercheurs permanents de l'Ecole est chargé de résoudre les problèmes liés à l'organisation pédagogique des enseignements. Le Conseil Scientifique propose les orientations pédagogiques et scientifiques de l'établissement, en tenant compte notamment de l'évolution du marché de travail et de l'adéquation des formations dispensées par rapport aux besoins des entreprises.

La figure 1 représente l'organigramme actuel de l'Ecole Nationale d'Informatique.

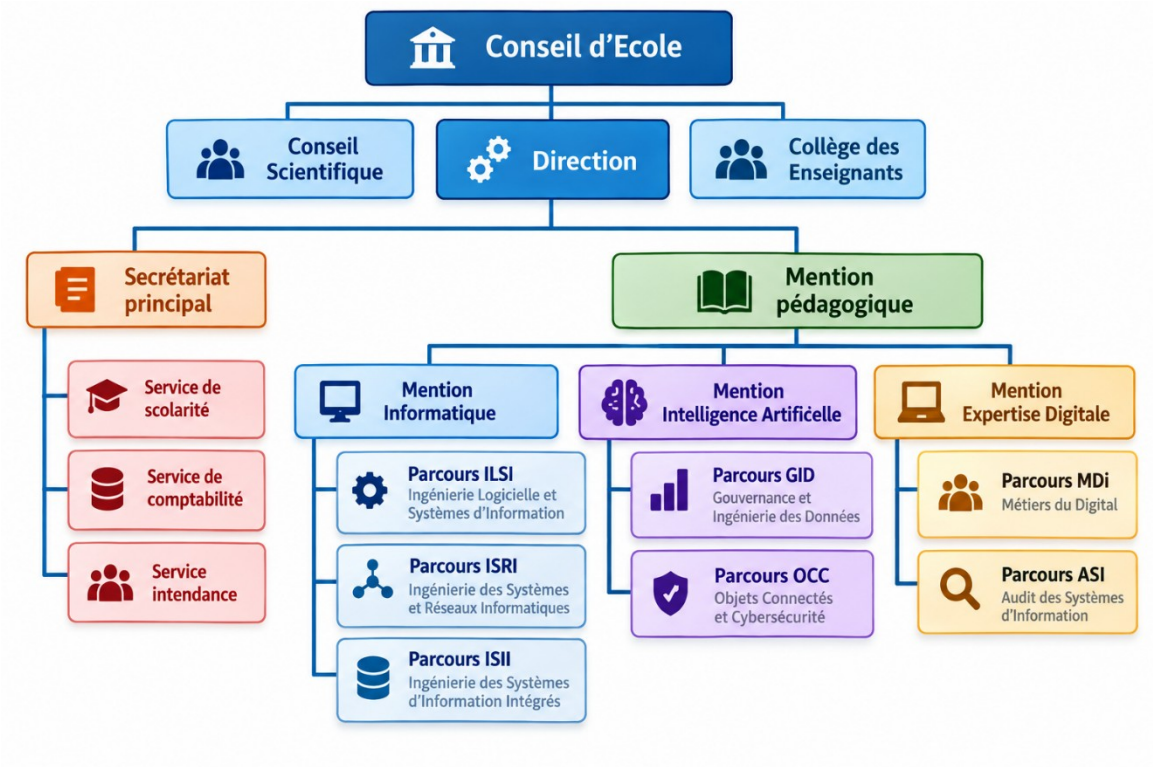


Figure 1: Organigramme de l'Ecole Nationale d'Informatique

4. Domaine de spécialisation

Les activités de formation et de recherche organisées à l'ENI portent sur les domaines suivants :

- Ingénierie logicielle et systèmes d'information
- Ingénierie des systèmes et réseaux informatiques
- Ingénierie des systèmes d'information intégrés
- Gouvernance et ingénierie des données
- Objets connectés et cybersécurité

- Métiers du digital et transformation numérique
- Audit des systèmes d'information
- Intelligence artificielle et ses applications.
- Modélisation informatique et mathématique des Systèmes complexes ;

Le tableau 1 décrit l'organisation du système de formation pédagogique de l'Ecole.

Tableau 1: Organisation du système de formation pédagogique de l'Ecole

Formation Théorique	Formation Pratique
• Enseignement théorique • Travaux dirigés • Travaux pratiques • Conférences	• Etude de cas • Travaux de réalisation • Projets/ Projets tutorés • Voyages d'Etudes • Stages en entreprise

5. Architecture des formations pédagogiques

Le recrutement des étudiants à l'ENI se fait uniquement par voie de concours d'envergure nationale en première année. Les offres de formation organisées à l'Ecole ont été validées par la Commission Nationale d'Habilitation (CNH). Au sein de l'ENI, il existe trois mentions et sept parcours. Le tableau 2 récapitule les mentions et les parcours au sein de l'Ecole :

Tableau 2: Mention et parcours au sein de l'ENI

Mention	Parcours
Informatique	Ingénierie logicielle et systèmes d'information (ILSI)
	Ingénierie des systèmes et réseaux informatiques (ISRI)
	Ingénierie des systèmes d'information intégrés (ISII)
Intelligence Artificielle	Gouvernance et Ingénierie de Données (GID)

	Objets Connectés et Cyber sécurités (OCC)
Expertise Digitale	Métiers du Digital (MDi)
	Audit des Systèmes d'Information (ASI)

La figure 2 représente l'architecture des études correspondant au système LMD.

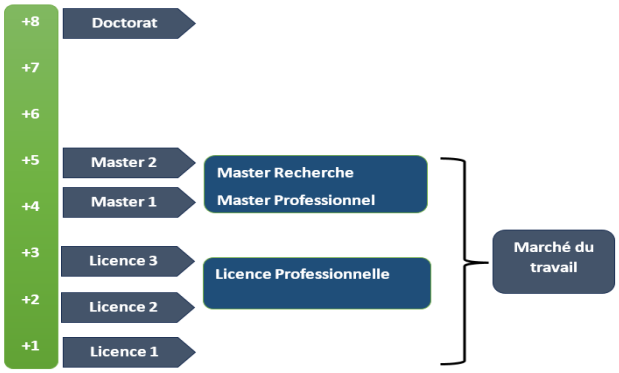


Figure 2: Architecture des études correspondant au système LMD

La licence peut avoir une vocation générale ou professionnelle. Le master peut avoir une vocation professionnelle ou de recherche. L'accès en première année de MASTER se fait automatiquement pour les étudiants de l'Ecole qui ont obtenu le diplôme de Licence Professionnelle.

Le tableau 3 illustre la liste des formations existantes à l'ENI.

Tableau 3: Liste des formations existantes à l'ENI

	FORMATION	
	LICENCE PROFESSIONNELLE	MASTER
Condition admission	Par voie de concours	Par voie de concours pour la mention IA
Condition d'accès	Bac de série C, D ou Technique	Être titulaire de licence professionnelle
Durée de Formation	3 ans	2 ans

Diplôme délivré	Diplôme de Licence Professionnelle	Diplôme de Master Professionnel Diplôme de Master Recherche

Le Master Recherche permet à son titulaire de poursuivre directement des études en doctorat et de s'inscrire directement dans une Ecole Doctorale.

Les étudiants diplômés de l'Ecole sont plutôt bien accueillis dans les instituts universitaires étrangères (Canada, Suisse, France, …)

6. Relation de l'ENI avec les organismes externes

Les stages effectués chaque année par les étudiants mettent l'Ecole en rapport permanent avec plus de 400 entreprises et organismes publics, semi-publics et privés, nationaux et internationaux. L'Ecole dispose ainsi d'un réseau d'entreprises, de sociétés et d'organismes publics et privés qui sont des partenaires par l'accueil en stage de ses étudiants, et éventuellement pour le recrutement après l'obtention des diplômes par ces derniers. Les compétences que l'Ecole cherche à développer chez ses étudiants sont l'adaptabilité, le sens de la responsabilité, du travail en équipe, le goût de l'expérimentation et l'innovation.

En effet, la vocation de l'ENI est de former des licenciés et des ingénieurs de niveau MASTER avec des qualités scientifiques, techniques et humaines reconnues, capables d'évoluer professionnellement dans des secteurs d'activité variés intégrant l'informatique. Les stages en milieu professionnel permettent de favoriser une meilleure adéquation entre les formations à l'Ecole et les besoins évolutifs du marché de l'emploi.

Parmi les sociétés, les entreprises et les organismes partenaires de l'Ecole, on peut citer : ACCENTURE Mauritius, AKATA Goavana, Air Madagascar, Ambre Associates, Airtel, Agence Universitaire de la Francophonie (AUF), AXIAN, B2B, Banque Centrale, , BIANCO, BlueLine, CNaPS, Bureau National de Gestion des Risques et des Catastrophes (BNGRC), CEDII-Fianarantsoa, Data Consulting, Central Test, Centre National Antiacridien, CNRE, COLAS, Direction Générale des Douanes, DLC, E-Tech Consulting, , FID, FIHARY Soft, FTM, GNOSYS, GENIUS AT WORK, Hello Tana, IBONIA, INGENOSIA, INSTAT, IOGA, JIRAMA, JOUVE, MADADEV, MAEP, MANAO, MEF, MEN, MESupRES, MFB, , MININTER, Min des Postes/Télécommunications et du Développement Numérique, NEOV MAD, Ny Havana, Madagascar National Parks, OMNITEC, ORANGE, OTME, PRACCESS,

QMM Fort-Dauphin, SG Madagasikara SMMC, SMMEC, SNEDADRS Antsirabe, Sénat, Société d'Exploitation du Port de Toamasina (SEPT), SOFTWELL, Strategy Consulting, TELMA, VIVETEC, Société LAZAN'I BETSILEO, WWF, UGD, ARATO, MANAO, MNDPT, NG ACADEMY.NG, Relia, Spoon Consulting …

7. Débouchés professionnels et diplômés

Les formations proposées par l'Ecole permettent aux diplômés d'être immédiatement opérationnels sur le marché du travail avec la connaissance d'un métier complet lié à l'informatique aux TIC.

L'Ecole apporte à ses étudiants un savoir-faire et un savoir-être qui les accompagnent tout au long de leur vie professionnelle. Elle a une vocation professionnalisante. Les diplômés en LICENCE et en MASTER issus de l'ENI peuvent faire carrière dans différents secteurs.

L'Ecole bénéficie aujourd'hui de 40 années d'expériences pédagogiques et de reconnaissance auprès des sociétés, des entreprises et des organismes. C'est une Ecole Supérieure de référence en matière informatique.

D'une manière générale, les diplômés de l'ENI n'éprouvent pas de difficultés particulières à être recrutés au terme de leurs études. Cependant, l'ENI recommande à ses diplômés de promouvoir l'entrepreneuriat en TIC et de créer des cybercafés, des SSII ou des bureaux d'études.

Le tableau 4 représente les débouchés éventuels selon la mention.

Tableau 4: Débouchés éventuels des jeunes diplômés

Mention	Débouchés professionnels
INFORMATIQUE	LSI : Développeur logiciel / développeur full-stack, Ingénieur en développement d'applications (desktop, web, mobile), Architecte logiciel, Intégrateur d'applications, Développeur d'API, Administrateur de bases de données (DBA), Ingénieur en conception de bases de données, Développeur SQL / PL-SQL, Spécialiste en migration et optimisation de bases de données, Data engineer (ingénieur en traitement de données), Analyste programmeur, Analyste fonctionnel, Chef de projet informatique, Scrum master / Product owner (en

	méthodologie Agile), Ingénieur QA (Quality Assurance), Testeur logiciel / testeur automatisé, Spécialiste maintenance applicative, Expert en refactoring et optimisation de code, Analyste de données (Data analyst), Concepteur d'outils décisionnels (BI), Ingénieur en Big Data, Machine Learning Engineer, Consultant fonctionnel ou freelance ISRI : Administrateur systèmes et réseaux, Ingénieur systèmes et réseaux, Administrateur systèmes Linux / Windows, Administrateur cloud (AWS, Azure, Google Cloud), Ingénieur virtualisation (VMware, Hyper-V, Proxmox), Administrateur sécurité (Security Administrator), Ingénieur cybersécurité, Analyste SOC (Security Operations Center), Spécialiste firewall / VPN, Responsable sécurité informatique (RSSI – avec expérience), Technicien support informatique (N2/N3), Technicien réseaux et télécoms, Ingénieur exploitation, Administrateur messagerie (Exchange, Microsoft 365), Technicien datacenter, Architecte réseaux, Architecte systèmes, Consultant infrastructure, Chef de projet systèmes et réseaux, Ingénieur DevOps, Administrateur Kubernetes / Docker, Cloud engineer, Spécialiste automatisation (Ansible, Terraform) ISII : ILSI et ISRI en même temps
INTELLIGENCE ARTIFICIELLE	GID : Développeur IA, Data Scientist, Entrepreneur IA, Consultant IA, Responsable marketing IA, Business Developer IA, Product Owner, Consultant en cybersécurité, Directeur de l'Innovation ; OCC : Analyste en cybersécurité, Ingénieur en cybersécurité, Spécialiste en réponse aux incidents, Architecte de sécurité, Consultant en cybersécurité, Auditeur en cybersécurité, Expert en sécurité des données, Responsable de la sécurité des informations, Chercheur en cybersécurité, Formateur en cybersécurité, Ingénieur IoT, Développeur IoT, Architecte IoT, Spécialiste en sécurité IoT, Analyste de données
EXPERTISE DIGITALE	MDi : Spécialiste en Marketing Digital, Gestionnaire de Contenu Web, Analyste UX/UI, Product Owner, Chef de Projet Digital, Stratège en Médias Sociaux, Analyste de Performance Digitale, Chef de Projet Digital, Stratège en Médias Sociaux, Concepteur-Rédacteur Web Analyste de Performance Digitale, Expert en Commerce Électronique, Expert en Commerce Électronique, Spécialiste SEO/SEM, Manager de Communauté (Community Manager), Conseiller en Stratégie Digitale, Responsable CRM (Customer Relationship Management), Spécialiste en Analytique et Intelligence d'Affaires (Business Intelligence), Consultant en Transformation Digitale, Responsable des Partenariats Digitaux, Spécialiste en Cybersécurité Marketing, Directeur de l'Innovation Digitale, Manager de l'E-réputation ASI : Auditeur des Systèmes d'Information, Consultant en Sécurité des Systèmes d'Information, Analyste de Risques IT, Chef de Projet SI, Responsable de la Conformité IT, Spécialiste en Cyber Forensics,

	Directeur de l'Audit et de la Sécurité des Systèmes d'Information, Expert en Gouvernance IT, Formateur ou Éducateur en Sécurité des Chercheur en Sécurité IT

8. Ressources humaines

Les ressources humaines sont citées ci-dessous selon leurs responsabilités :

- Directeur de l'Ecole : Monsieur MAHATODY Thomas, Professeur
- Responsable de la Mention « Informatique » : Monsieur RALAIVAO Jean Christian, Maître de Conférences
- Responsable de la Mention « Intelligence Artificielle » : Monsieur DIMBISOA William Germain, Maître de Conférences
- Responsable de parcours « ILSI » : Madame RATIANANTITRA Volatiana Marielle, Maître de Conférences
- Responsable de parcours « ISRI » : Monsieur RAZAFINDRAIBE Marolahy Alix, Assistant d'Enseignement Supérieur et de Recherche
- Responsable de parcours « ISII » : Monsieur RAZAFINDRAMONJA Clément Aubert, Assistant d'Enseignement Supérieur et de Recherche
- Responsable de parcours « GID » : Madame RAKOTOASIMBAHOAKA Antsa Cyprienna, Maître de Conférences
- Responsable de parcours « OCC » : Monsieur GILANTE Gesazafy, Assistant d'Enseignement Supérieur et de Recherche

L'ENI compte seize (16) enseignants permanents dont un (01) Professeur Titulaire, deux (02) Professeurs, Neuf (09) Maîtres de Conférences, Quatre (04) Assistants d'Enseignement Supérieur et de Recherche, plusieurs enseignants vacataires et personnel administratif.

Chapitre 2. Présentation de l’établissement d’accueil

Ce chapitre présentera l’entreprise Miro K.E., sous divers angles. Nous commencerons par les informations générales de l’entreprise, puis aborderons ses missions, son organisation, ainsi que ses partenariats. Enfin, ses valeurs fondamentales.

2.1.Information de l’entreprise

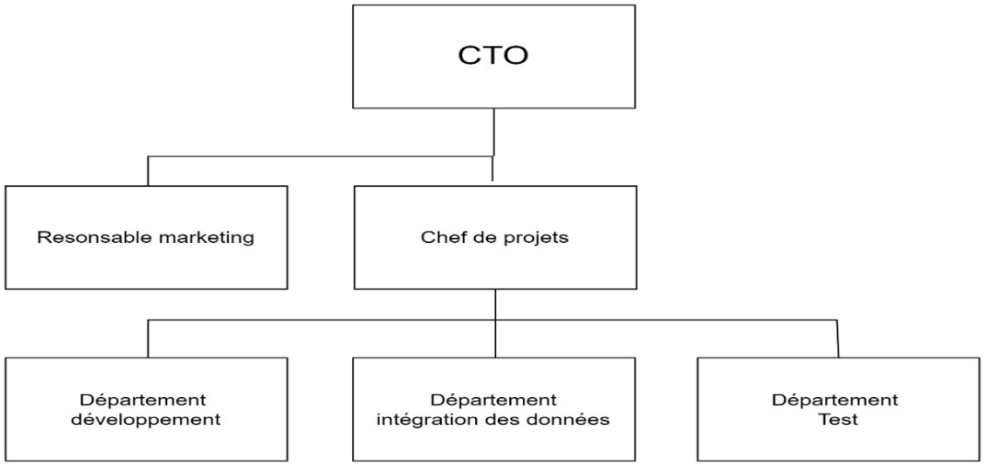
Miro K.E. dispose de deux implantations : l’une à Ambohidratrimo, Madagascar, et l’autre à Dubaï, aux Émirats Arabes Unis. La localisation et le contact de la société sont montrés dans le tableau 5.

Tableau 5.Information de l'entreprise MIRO KE

Etat	United Arab Emirates	Madagascar
Adresse postal	Dubai Silicon Oasis Dubai, United Arab Emirates	Po Box 42, Ambohidratrimo Antananarivo 105 Madagascar
Email	c.rakotondratsima@MiroK.E.com	lyg.rakotondratsima@Miro K.E.groupe-ankarana.com

2.2.Organisation du MIRO KE

La figure 3 présente l’organigramme actuel de l’Entreprise**.**



2.3.Missions

MIRO Knowledge Engineering ou MIRO K.E est une société spécialisée dans les systèmes et solutions pour les entreprises et les organisations publiques ou privées. MIRO KE a ses principaux clients basés aux Emirats Arabes Unis, dans un marché très concurrentiel ou les grands éditeurs Américains et Européens se retrouvent avec les éditeurs en provenance de l'Inde. MIRO KE offre des solutions clés en main en ayant une maitrise du cycle complet de production ainsi que tous les composants logiciels de la solution, incluant le système d'exploitation, le système de gestion de base des données, le serveur d'application et le client léger.

2.4.Partenariat

Miro K.E. fait partie des « Silver Partners » d'Odoo. Les « Silver Partners » sont expérimentés dans la mise en œuvre des projets d'Odoo. Ils ont une relation étroite avec Odoo et ils ont prouvé qu'ils peuvent mettre en œuvre des solutions Odoo efficacement. Miro K.E. envisage des partenariats similaires avec Talend, Pentaho et Jaspersoft.

2.5.Valeurs fondamentales

Une combinaison de clarté, de simplicité, qualité, fiabilité, justice et solidité. Ce sont les principes avec lesquels Miro K.E. exerce ses activités, de la conception d'une solution à sa livraison.

- Clarté :

  Miro K.E. fournit des solutions et des lignes directrices claires pour ses clients.

- Simplicité :

  Les produits livrés sont simples à utiliser et intuitifs pour les utilisateurs. Miro K.E. est en mesure de disséquer les processus complexes et les problèmes de données.

- Qualité :

  Miro K.E. porte une attention particulière à la qualité de ses produits, de ses services, de son travail, des techniques et de l'information. L'équipe de Miro K.E. se réunit régulièrement pour partager des connaissances et des meilleures pratiques.

- Fiabilité :

  Miro K.E. offre des solutions pour répondre aux besoins actuels des clients et donne une garantie de 1 an après sa livraison sur ses produits livrés.

- Équité :

Miro K.E. propose des solutions solides et complètes, répondant aux besoins des clients de la meilleure façon possible et en tenant compte des contraintes. Elle considère ses clients comme des partenaires.

- Solidité :

  Toutes les solutions proposées par Miro K.E. passent par des tests approfondis avant la livraison. Miro K.E. reste déterminée à corriger le code jusqu'à ce qu'il soit parfait.

Miro K.E. valorise la qualité technique des solutions qu'il propose et privilégie avec cela :

- La simplicité d'utilisation
- La réduction au maximum pour ses clients, du coût total de possession, incluant la spécification, le suivi du projet, le test, vérification, validation, le déploiement, la formation, la maintenance et l'évolution.

Chapitre 3. Description du projet

Ce chapitre présente de manière exhaustive le cadre fonctionnel, technique et organisationnel du projet. Il expose la formulation précise de la problématique, les objectifs poursuivis, les besoins fonctionnels et non fonctionnels, l'inventaire des ressources humaines, matérielles et logicielles mobilisées, les résultats attendus, les perspectives stratégiques d'extension (notamment le protocole universel MCP et le module de Code Intelligence) ainsi que la planification prévisionnelle des travaux.

3.1. Formulation du projet

Le projet s'intitule : « Conception, réalisation et intégration d'une intelligence artificielle privée dans Odoo 17 ». Il répond à une demande formelle exprimée par la direction technique de la société MIRO K.E., intégrateur et Silver Partner Odoo. L'entreprise constate qu'en dépit de la richesse fonctionnelle d'Odoo 17, les utilisateurs finaux (collaborateurs administratifs, commerciaux, gestionnaires de stocks, techniciens) éprouvent des difficultés régulières à naviguer dans l'arborescence complexe des menus, à synthétiser rapidement des volumes importants de données et à exploiter efficacement la documentation interne.

Parallèlement, la direction proscrit formellement le recours aux modèles d'IA hébergés sur le cloud public pour des impératifs stricts de confidentialité, de secret d'affaires et de conformité réglementaire, interdisant tout transfert de données clients, financières ou RH vers des infrastructures tierces.

Pour répondre à cette double exigence, le projet adopte une architecture tripartite modulaire :

- **L'addon Odoo 17 « miro_ai » :** module d'extension Odoo natif développé avec le framework JavaScript OWL (Odoo Web Library) et Python Odoo. Il s'intègre harmonieusement dans l'interface web (bouton systray, tiroir de chat interactif, rendu markdown et citations de sources), intègre un contrôleur sécurisé avec signature HMAC et clés temporaires, et maintient un journal d'audit complet des interactions (miro.ai.log).
- **Le serveur middleware d'orchestration IA :** service backend indépendant développé avec FastAPI et LangGraph, chargé d'analyser les requêtes en langage naturel, de déterminer la route d'exécution optimale et de piloter les agents spécialisés.
- **Les moteurs d'exécution spécialisés :** d'une part, un moteur RAG hybride (dense BGE-M3 et lexical BM25 avec fusion RRF) assurant la recherche documentaire précise ; d'autre part, un agent de lecture ORM Odoo temps réel interrogeant directement la base de gestion sous les droits et privilèges stricts de l'utilisateur connecté sans recours à sudo().

Le périmètre fonctionnel validé et démontré dans le présent mémoire se focalise sur l'intégration du module miro_ai, l'interrogation documentaire fiable et la consultation temps réel sécurisée de l'ORM Odoo.

3.2. Objectif et besoins de l’utilisateur

3.2.1. Objectif général

L'objectif fondamental de ce projet est de concevoir, développer et intégrer dans Odoo 17 une solution d'intelligence artificielle conversationnelle souveraine et privée (miro_ai et son backend dédié), garantissant la stricte confidentialité des données d'entreprise, le respect absolu des privilèges applicatifs RBAC et une assistance performante sur la documentation et les données de gestion.

3.2.2. Besoins fonctionnels

Les fonctionnalités attendues du système se structurent selon les rôles des utilisateurs :

Collaborateurs et gestionnaires métier :

- Accéder à l'assistant conversationnel directement depuis n'importe quelle vue de travail d'Odoo 17 (icône systray dans la barre supérieure et volet latéral).
- Poser des questions en langage naturel relatives aux procédures internes, politiques d'entreprise et manuels d'utilisation (RAG documentaire).
- Consulter l'état des données opérationnelles de l'ERP en temps réel (ex. : devis en attente, factures impayées, niveaux de stock, coordonnées de partenaires).
- Visualiser des réponses enrichies avec mise en forme dynamique (Markdown, listes, tableaux), badges de niveau de confiance et citation précise des sources documentaires ou identifiants d'enregistrements.
- Consulter l'historique des conversations récentes et reprendre un fil d'échange antérieur.

Administrateurs du système :

- Configurer les paramètres de connexion du module miro_ai dans les paramètres généraux d'Odoo (URL du backend, clé secrète HMAC, délai de réponse timeout).
- Activer ou restreindre les domaines de compétences métier de l'assistant (Ventes, Achats, Stocks, Comptabilité, Support technique).
- Consulter le journal d'audit complet (miro.ai.log) retraçant les requêtes, les temps de réponse en millisecondes, les sources utilisées et les éventuelles anomalies.
- Gérer l'indexation et la mise à jour des bases documentaires vectorielles.

3.2.3. Besoins non fonctionnels

Les exigences qualitatives et techniques indispensables sont les suivantes :

- **Sécurité et gouvernance stricte des données :** Principe du moindre privilège : chaque requête ORM doit être exécutée strictement sous l'identité de l'utilisateur connecté (uid), en appliquant les listes de contrôle d'accès (ir.model.access) et les règles d'enregistrement multi-

sociétés (ir.rule). Absence totale d'élévation de privilèges : interdiction d'utiliser sudo() ou SUPERUSER_ID pour contourner les droits. Authentification des flux inter-services par HMAC SHA-256.

• **Confidentialité et souveraineté locale :** Traitement intégral des requêtes et inférence des modèles de langage en local (sur l'infrastructure privée de Miro K.E.). Aucune donnée ne transite par des serveurs tiers.

• **Performance et réactivité :** Temps de réponse optimisé (< 2 secondes pour l'analyse et la recherche documentaire). Architecture asynchrone par scrutation de tâches (job polling) dans l'addon miro_ai afin de libérer immédiatement les processus web d'Odoo.

• **Ergonomie et fidélité à l'écosystème Odoo :** Développement de l'interface en JavaScript OWL 2.0 respectant la charte graphique et l'expérience utilisateur standard d'Odoo 17.

• **Tolérance aux pannes :** Mécanisme de basculement automatique (failover) au niveau de miro_ai en cas d'indisponibilité momentanée d'une instance du backend IA.

3.3. Moyens nécessaires à la réalisation du projet

La réalisation de ce travail de recherche et d'ingénierie a mobilisé des ressources humaines, matérielles et logicielles ciblées.

3.3.1. Moyens humains

Le projet implique une collaboration étroite entre l'institution académique et l'entreprise d'accueil, comme l'illustre le Tableau 6.

Tableau 6. Moyens humains

Rôle / Fonction	Effectif	Attribution et responsabilités
Encadreur pédagogique (ENI)	1	Orientation méthodologique, validation scientifique des modèles et suivi académique.
Encadreur professionnel (Miro K.E.)	1	Définition des besoins métier Odoo, cadrage applicatif et accompagnement technique.
Étudiant-chercheur / Développeur	1	Analyse, conception architecturale, développement du module miro_ai, du backend IA et intégration.

3.3.2. Moyens matériels

L'exécution locale des modèles de fondation et le développement conjoint de la solution requièrent des équipements informatiques adéquats, détaillés dans le Tableau 7.

Tableau 7. Moyens matériels

Désignation	Processeur (CPU)	Mémoire vive (RAM)	Stockage	Accélération GPU	Affectation
Station de travail de développement	Intel Core i7 / AMD Ryzen 7 (8 cœurs, 16 threads)	32 Go DDR4/DDR5	SSD NVMe 1 To	NVIDIA RTX (8 Go VRAM)	Développement logiciel (miro_ai, FastAPI), tests unitaires, containerisation.
Serveur d'inférence IA	Serveur x86_64 multi-cœurs	64 Go DDR4 ECC	SSD NVMe 2 To	NVIDIA RTX / Tesla (16 à 24 Go VRAM)	Déploiement et exécution des LLM locaux (Ollama/vLLM) et vector store.

3.3.3. Moyens logiciels

L'ensemble de la pile logicielle retenue pour le projet est résumé dans le Tableau 8.

Tableau 8. Moyens logiciels

N°	Désignation	Version	Rôle et utilisation
1	Odoo ERP	17.0 Community / Enterprise	Plateforme de gestion intégrée cible hébergeant le module addon miro_ai.
2	PostgreSQL	16.x	Système de gestion de base de données relationnelle d'Odoo.
3	Python	3.10 / 3.11	Langage principal du module Odoo miro_ai et du backend d'IA.
4	Odoo Web Library (OWL)	OWL 2.0	Framework JavaScript réactif utilisé pour l'interface du module miro_ai.
5	Ollama / vLLM	0.3+ / 0.6+	Moteurs d'inférence et de service pour modèles de fondation locaux.
6	Modèles de langage (LLM)	Qwen2.5 (7B/14B) / Llama 3 (8B)	Modèles de base pour le raisonnement, l'orchestration et la génération.
7	Modèle d'embedding & Rerank	BGE-M3 (BAAI) / FlagEmbedding	Génération de représentations vectorielles denses et multi-facettes.

8	Framework Agentique	LangGraph / LangChain	Modélisation des flux décisionnels sous forme de graphes d'états.
9	FastAPI & Pydantic	0.110+	Exposition des services d'IA sous forme d'API REST asynchrones.
10	Docker & Docker-Compose	26+	Conteneurisation et reproductibilité de l'environnement IA.
11	Git & GitHub	2.40+	Gestion des versions de code source et collaboration.
12	Visual Studio Code	1.94+	Environnement de développement intégré (IDE) principal.

3.4. Résultats attendus

Au terme de la mise en œuvre, le projet doit aboutir aux résultats concrets suivants :

• **Module addon Odoo 17 natif (« miro_ai ») :** Fourniture d'un module propre, modulaire et directement installable via le gestionnaire d'applications standard d'Odoo, intégrant les vues, la sécurité, les contrôleurs et la traçabilité des échanges.

• **Interface utilisateur réactive en OWL :** Intégration ergonomique dans le client web d'Odoo via un composant OWL offrant un volet de dialogue accessible en permanence sans altérer l'expérience utilisateur standard.

• **RAG hybride haute fidélité :** Moteur d'extraction documentaire combinant recherche sémantique dense et lexicale, offrant un taux de rappel supérieur à 90 % et éliminant les hallucinations grâce à la citation rigoureuse des sources documentaires.

• **Agent ORM temps réel sécurisé :** Restitution instantanée d'indicateurs de gestion tout en garantissant l'étanchéité absolue des données entre utilisateurs et entre sociétés.

• **Souveraineté et confidentialité totales :** Traitement en circuit fermé attesté par l'absence d'émission de requêtes vers des clouds tiers lors des interactions utilisateur.

3.5. Perspectives d'évolution du projet

Le périmètre opérationnel validé dans le cadre de ce mémoire (l'addon Odoo miro_ai, le RAG documentaire et la lecture ORM sécurisée) pose les fondations d'un écosystème d'IA d'entreprise complet. Deux axes stratégiques majeurs constituent les perspectives d'évolution immédiates du système :

3.5.1. Standardisation des outils via un serveur MCP (Model Context Protocol)

L'évolution rapide des architectures agentiques fait émerger des standards d'interconnexion universels. Le protocole MCP (Model Context Protocol), introduit comme standard ouvert par l'industrie, vise à unifier la manière dont les modèles d'IA interagissent avec les sources de données et les outils externes de façon standardisée, modulaire et hautement sécurisée.

Dans cette perspective, il est prévu de développer un serveur MCP natif dédié à Odoo 17. Ce serveur exposera dynamiquement les métadonnées de l'ERP, les schémas de modèles et les outils de consultation sous une interface MCP normalisée. Ainsi, n'importe quel agent ou environnement IA autorisé (qu'il s'agisse d'un client bureautique, d'un IDE assisté par IA ou d'un système multi-agents autonome) pourra interroger et exploiter l'ERP Odoo sans adaptateur propriétaire supplémentaire, tout en conservant le contrôle d'accès strict hérité du contexte de session utilisateur.

3.5.2. Module de Code Intelligence et analyse statique des modèles Odoo

La seconde perspective majeure réside dans le déploiement du module de Code Intelligence, spécifiquement destiné aux développeurs et administrateurs techniques de Miro K.E. Ce module s'appuie sur une analyse syntaxique abstraite (AST — Abstract Syntax Tree) et sur la construction d'un graphe de connaissances sémantique de plus de 15 000 nœuds et relations, modélisant avec précision l'intégralité de l'écosystème Odoo déployé (modèles Python, champs, vues XML, règles de sécurité ir.model.access, actions de serveur et dépendances croisées de modules).

Grâce à cette représentation structurelle approfondie, le module de Code Intelligence offrira des capacités inédites :

- **Explication de workflows complexes :** Capacité à restituer et commenter visuellement la chaîne d'héritage d'un modèle et les déclencheurs d'actions automatisées.
- **Diagnostic des blocages métier :** Identification rapide des conflits d'héritage, des dépendances circulaires ou des restrictions d'accès bloquant un utilisateur.
- **Audit statique de sécurité automatisé :** Détection préventive d'appels non sécurisés à sudo(), de requêtes SQL brutes non paramétrées ou d'oublis de déclarations de règles d'accès avant la mise en production.
- **Assistance à la migration de versions :** Accompagnement intelligent lors de la montée de version majeure d'Odoo (par exemple le passage de la v16 vers la v17) en cartographiant les méthodes et champs dépréciés.

3.6. Chronogramme de travail

La démarche méthodologique adoptée pour mener à bien le projet s'est échelonnée sur plusieurs phases successives, résumées dans le Tableau 9.

Tableau 9. Chronogramme prévisionnel de réalisation du projet

Phase	Intitulé des travaux	Période indicative	Livrables associés
Phase 1	Analyse des besoins, état de l'art et étude de faisabilité de l'IA locale	Mois 1	Rapport d'analyse préalable, benchmark comparatif des LLM et architectures RAG.
Phase 2	Conception architecturale, modélisation de la sécurité et des flux	Mois 2	Spécifications techniques, diagrammes UML, matrice des flux et schéma de sécurité.
Phase 3	Développement du moteur RAG hybride et de l'agent ORM sécurisé	Mois 3	Pipeline d'indexation/rerank BGE-M3, endpoints FastAPI, connecteurs ORM sécurisés.
Phase 4	Développement du module Odoo 17 (miro_ai) et intégration UI OWL	Mois 4	Addon Odoo miro_ai (composants OWL, contrôleurs, journal d'audit miro.ai.log).
Phase 5	Campagne de tests, benchmarking des performances et évaluation	Mois 5	Jeux d'essais, métriques de rappel/latence, protocole d'évaluation et validation fonctionnelle.
Phase 6	Rédaction finale du mémoire de Master et préparation de la soutenance	Mois 6	Manuscrit du mémoire de Master ENI complet, supports de présentation.

PARTIE II: ANALYSE ET CONCEPTION

Chapitre 4. Analyse préalable

L'intégration réussie d'une solution d'intelligence artificielle au sein d'un Progiciel de Gestion Intégré exige une analyse rigoureuse des processus organisationnels existants et des flux informationnels. Ce chapitre présente l'analyse préalable du projet en trois étapes majeures : l'étude de l'existant au sein de l'entreprise d'accueil MIRO K.E. et de son écosystème d'utilisateurs Odoo 17, la critique méthodique identifiant les verrous technologiques et les risques de sécurité (notamment l'usage non maîtrisé d'IA externes), et enfin la conception avant-projet détaillant la formulation des architectures envisageables, la justification de la solution retenue ainsi que le choix des démarches méthodologiques et des outils de modélisation.

4.1. Analyse de l’existant

4.1.1. Organisation actuelle et processus métier

L'entreprise MIRO K.E. exploite la plateforme Odoo 17 pour piloter ses activités internes et déploie cette même solution auprès de ses clients d'horizons variés (négoce, services, industrie, distribution). Au sein de cette organisation, le Progiciel de Gestion Intégré concentre les opérations critiques quotidiennes à travers plusieurs modules métiers interconnectés : gestion commerciale et devis, facturation et comptabilité analytique, gestion des stocks et réapprovisionnements, gestion des relations clients (CRM) et assistance technique.

L'analyse opérationnelle révèle deux flux de traitement informationnel majeurs, aujourd'hui exécutés de manière cloisonnée et manuelle :

- **La recherche documentaire et procédurale :** Les collaborateurs (consultants d'intégration, comptables, gestionnaires administratifs) doivent fréquemment consulter les guides méthodologiques, les fiches techniques d'implémentation et les règles de gestion spécifiques aux configurations déployées. Ces ressources documentaires sont réparties sous divers formats statiques (manuels PDF, notes de versions, dépôts de documentation Markdown et wikis internes). La recherche d'une procédure précise requiert une navigation manuelle chronophage et se révèle souvent infructueuse face à l'abondance des référentiels.
- **L'interrogation des données transactionnelles de l'ERP :** Lorsqu'un responsable commercial ou un gestionnaire de stock souhaite obtenir un indicateur de gestion (par exemple, le montant cumulé des commandes en cours d'un client, la liste des factures impayées échues ou la disponibilité immédiate d'un article multi-sociétés), il est contraint d'ouvrir le menu dédié, de manipuler des filtres de recherche complexes, de basculer entre plusieurs vues (liste, formulaire, pivot) ou d'exporter les enregistrements sous tableur externe afin d'opérer des regroupements manuels.

Sur le plan organisationnel, les catégories de personnel impliquées dans ces traitements regroupent la Direction technique et commerciale, les consultants intégrateurs Odoo, les gestionnaires de paie et de facturation, ainsi que les équipes de support client de Miro K.E.

4.1.2. Inventaire des moyens matériels et logiciels

L'infrastructure technique actuellement en exploitation au sein de Miro K.E. pour supporter les environnements Odoo 17 est synthétisée dans le Tableau 10.

Tableau 10. Inventaire des infrastructures matérielles et logicielles existantes chez Miro K.E.

Catégorie	Composant / Équipement	Spécifications techniques	Rôle opérationnel
Matériel	Serveurs d'hébergement Odoo	Processeurs multi-cœurs x86_64, 32 Go RAM, SSD NVMe	Hébergement des instances de test, préproduction et production.
Matériel	Postes de travail collaborateurs	PC portables et fixes Intel Core i5/i7, 16 Go RAM	Postes clients de consultation, de gestion administrative et de développement.
Logiciel	Système d'exploitation serveur	Linux Debian 12 / Ubuntu Server 22.04 LTS	Environnement sécurisé d'exécution des services d'entreprise.
Logiciel	Progiciel de Gestion Intégré	Odoo 17.0 (Community et Enterprise)	Plateforme centrale de gestion d'entreprise (CRM, Vente, Stock, Compta).
Logiciel	Système de Gestion de BDD	PostgreSQL 14 / 16	Stockage persistant des données relationnelles et transactionnelles.
Logiciel	Navigateurs Web clients	Google Chrome, Mozilla Firefox (dernières versions)	Accès web des utilisateurs aux interfaces de l'ERP.
Logiciel	Outils d'assistance existants	Recherche textuelle native d'Odoo, tableurs	Recherche élémentaire par sous-chaînes de caractères sans compréhension sémantique.

4.2. Critique de l’existant

L'évaluation approfondie du fonctionnement actuel met en lumière des atouts structurels indéniables, mais révèle également des goulots d'étranglement critiques ainsi que des risques de sécurité majeurs.

Points forts de l'organisation actuelle :

• **Centralisation et cohérence des données :** L'utilisation d'Odoo 17 élimine la redondance des enregistrements en regroupant clients, commandes, stocks et factures dans une base unique et normalisée.

• **Sécurité applicative native robuste :** Odoo dispose d'un système de contrôle d'accès éprouvé (RBAC), combinant autorisations de modèles (ir.model.access) et filtres dynamiques au niveau des lignes (ir.rule), garantissant un cloisonnement strict entre utilisateurs et sociétés.

• **Architecture web moderne :** Le framework JavaScript OWL 2.0 offre une interface utilisateur réactive, composable et modulaire adaptée aux environnements web d'entreprise.

Points faibles et vulnérabilités identifiés :

• **Surcharge cognitive et perte de productivité :** L'arborescence étendue des menus d'Odoo 17 rend complexe l'accès rapide aux informations synthétiques pour les utilisateurs non spécialistes, générant des lenteurs et des interruptions répétées du travail opérationnel.

• **Incapacité de la recherche native à traiter le langage naturel :** La recherche textuelle standard d'Odoo s'effectue par correspondance stricte de chaînes (opérateur ILIKE SQL). Elle s'avère totalement inopérante pour interpréter des questions conceptuelles, des formulations synonymes ou des requêtes métier complexes.

• **Fragmentation des connaissances documentaires :** Les documents techniques et procéduraux sont dissociés des flux opérationnels de l'ERP, privant les utilisateurs d'une assistance contextuelle immédiate lors de l'exécution de leurs tâches.

• **Risque critique de fuite de données et « Shadow AI » :** Face au besoin quotidien de synthétiser des textes ou d'analyser des situations complexes, certains collaborateurs ont recours à des assistants IA grand public en mode cloud (ChatGPT, Claude, etc.) en y injectant des extraits de données financières, clients ou RH. Cette pratique expose l'entreprise à des violations graves du secret commercial et des obligations légales de protection des données (loi n° 2014-038 à Madagascar et RGPD).

• **Absence d'étanchéité des privilèges dans les outils externes :** Aucun outil conversationnel généraliste externe n'est capable de respecter le contexte de session, l'identifiant strict de l'utilisateur (uid) et les règles multi-sociétés d'Odoo, ce qui interdit tout interfaçage direct sans supervision de sécurité.

4.3. Conception avant-projet

4.3.1. Proposition des solutions

Pour répondre aux limites identifiées et doter l'entreprise d'une assistance intelligente sécurisée, trois orientations architecturales ont été analysées :

• **Solution 1 : Intégration d'un service d'IA SaaS sur Cloud public (ex. API OpenAI GPT-4 / Claude) :** Cette approche consiste à développer un connecteur Odoo expédiant les questions des utilisateurs et des extraits de la base de données vers des serveurs hébergés par des fournisseurs tiers. Si elle bénéficie d'une puissance de calcul immédiate sans investissement matériel local, elle présente des risques rédhibitoires : fuite de données d'entreprise sensibles hors du périmètre souverain, coûts récurrents élevés indexés sur la consommation de jetons (tokens), dépendance absolue vis-à-vis d'une connexion Internet stable et impossibilité de garantir l'absence d'élévation de privilèges au niveau de l'ORM Odoo.

• **Solution 2 : Maintien du statu quo avec optimisation des outils manuels existants :** Cette solution préconise de former davantage les utilisateurs à l'ergonomie standard d'Odoo et de développer des tableaux de bord statiques supplémentaires. Bien qu'elle n'engendre aucun investissement matériel, elle ne résout ni la fracture cognitive liée à la multiplicité des menus, ni la fragmentation documentaire, et n'endigue en rien le recours informel au « Shadow AI ».

• **Solution 3 : Conception et déploiement d'une plateforme d'IA privée souveraine intégrée (Addon miro_ai + Backend FastAPI/LangGraph + LLM local + RAG hybride + ORM sécurisé) :** Cette architecture déploie une solution d'intelligence artificielle 100 % hébergée sur l'infrastructure privée de Miro K.E. Elle intègre un module addon Odoo 17 (miro_ai) conçu en JavaScript OWL pour l'IHM et des contrôleurs asynchrones HMAC, couplé à un middleware d'orchestration FastAPI/LangGraph opérant des modèles open-weight locaux (Ollama/vLLM, BGE-M3). Le moteur RAG hybride résout l'accès à la documentation avec citation des sources, tandis que l'agent ORM interroge l'ERP sous l'identité stricte du demandeur (uid) en respectant rigoureusement les droits RBAC sans jamais exécuter de sudo().

4.3.2. Analyse comparative et solution retenue

Le Tableau 11 présente une analyse comparative multicritères des trois solutions envisagées afin d'éclairer la décision stratégique.

Tableau 11. Analyse comparative des solutions architecturales envisagées

Critères d'évaluation	Solution 1 : IA Cloud SaaS	Solution 2 : Statu Quo	Solution 3 : IA Privée Souveraine (miro_ai)
Souveraineté et confidentialité des données	Faible (données transmises à des tiers)	Moyenne (données internes mais risque Shadow AI)	Maximale (traitement 100% on-premise en circuit fermé)
Respect du contrôle d'accès Odoo (RBAC)	Complexe et risqué (filtrage difficile en amont)	Natif (menus Odoo standard)	Strict (héritage dynamique de l'uid et des ir.rule sans sudo)
Modèle économique et coûts récurrents	Coûteux (abonnement et facturation aux tokens)	Nul (aucun coût d'IA additionnel)	Maîtrisé (amortissement du serveur matériel, zéro coût token)

Ergonomie et intégration dans l'ERP	Moyenne (widget déconnecté ou externe)	Faible (navigation complexe manuelle)	Excellente (composant natif OWL dans le systray d'Odoo 17)
Résistance aux hallucinations métier	Faible (modèle généraliste sans ancrage)	Non applicable	Élevée (RAG hybride avec citations et validation ORM)
Indépendance réseau et disponibilité	Dépendance critique à la bande passante	Totale en réseau local	Totale (opère même en cas de coupure Internet)

Au vu de cette analyse multicritères, la Solution 3 est formellement retenue. Elle est la seule à concilier les exigences absolues de souveraineté des données, de respect strict de la politique de sécurité RBAC d'Odoo, d'absence totale de coûts récurrents à l'usage et d'intégration fluide au poste de travail des utilisateurs grâce à l'addon miro_ai.

4.3.3. Méthodes de conception et outils retenus

La réussite d'un projet d'ingénierie logicielle couplant un ERP d'entreprise et des composants d'intelligence artificielle requiert une démarche méthodologique rigoureuse et des choix technologiques ciblés.

Démarche de conduite de projet : Processus Agile SCRUM

Contrairement aux cycles séquentiels rigides (tels que le cycle en V ou la méthode MERISE) qui fixent l'intégralité des spécifications avant le début du développement, le développement de systèmes à base d'agents IA et de RAG requiert une approche hautement itérative. Les réglages des modèles d'embedding, le seuillage de pertinence et la conception des prompts nécessitent des validations expérimentales rapprochées. C'est pourquoi la méthodologie Agile SCRUM a été adoptée, organisée en sprints de deux semaines jalonnés de démonstrations fonctionnelles auprès de l'équipe technique de Miro K.E. Le Tableau 12 confronte les caractéristiques de ces deux approches.

Tableau 12. Analyse comparative de la méthode Agile SCRUM et du cycle séquentiel (MERISE / Cascade)

Dimension méthodologique	Cycle séquentiel (MERISE / Cascade)	Démarche Agile SCRUM (Retenue)
Gestion des exigences	Spécifications exhaustives figées dès le démarrage	Product Backlog dynamique et priorisé selon la valeur métier
Adaptabilité aux aléas de l'IA	Rigide face aux imprévus de performance des modèles	Forte réactivité grâce aux revues de sprint régulières
Visibilité des livrables	Validation tardive lors de la phase de recette finale	Incréments logiciels testables livrés à chaque sprint
Implication des parties prenantes	Ponctuelle (au début et à la livraison finale)	Continue (collaboration étroite avec les encadreurs)

Langage de modélisation retenu : UML 2.5

Le langage de modélisation unifié UML (Unified Modeling Language), standard international de l'OMG, a été choisi pour concevoir et documenter les différentes facettes du système. Dans le cadre de ce projet, les diagrammes suivants seront particulièrement exploités :

- **Le diagramme de cas d'utilisation :** pour formaliser les interactions entre les utilisateurs finaux, les administrateurs et l'assistant IA miro_ai.
- **Les diagrammes de séquence :** pour expliciter le flux temporel asynchrone des requêtes, depuis le clic de l'utilisateur dans l'IHM OWL jusqu'à la consultation de l'ORM Odoo via FastAPI et la restitution finale.
- **Le diagramme de classes et de paquetages :** pour structurer l'architecture modulaire du backend (nœuds LangGraph, modèles de configuration Odoo, contrôleurs de sécurité).
- **Le diagramme de déploiement :** pour illustrer la topologie réseau, l'isolation des conteneurs Docker et les liaisons inter-processus sécurisées par HMAC.

Analyses comparatives des composants technologiques clés

Pour chaque couche critique de l'architecture, une sélection rigoureuse d'outils et de bibliothèques a été opérée, comme le détaillent les tableaux comparatifs ci-après :

Tableau 13. Analyse comparative des moteurs d'inférence LLM locaux

Moteur d'inférence	Points forts	Limites	Décision et usage
Ollama	Déploiement simple, gestion unifiée des modèles GGUF, excellente compatibilité	Optimisation multi-requêtes simultanées limitée	Retenu pour le prototypage rapide et l'environnement de développement
vLLM	Haut débit, gestion avancée de la mémoire (PagedAttention), idéal pour production	Nécessite des GPU dédiés avec environnement CUDA natif	Retenu pour le déploiement serveur de production à forte charge
HuggingFace TGI	Excellente intégration à l'écosystème HF, support de la quantification	Configuration complexe et empreinte mémoire importante	Écarté au profit d'Ollama et vLLM plus modulaires

Tableau 14. Analyse comparative des stratégies de recherche documentaire pour le RAG

Approche RAG	Comportement sur le vocabulaire métier	Comportement conceptuel	Décision et justification
Vectoriel dense pur (Dense-only)	Faible : peine sur les codes d'articles et termes techniques rares	Excellent : capte les synonymes et analogies sémantiques	Insuffisant pour la précision chirurgicale requise par un ERP

Lexical pur (BM25 seul)	Excellent sur les identifiants exacts et codes comptables	Nul : incapable d'interpréter des reformulations ou périphrases	Insuffisant face aux formulations libres des utilisateurs
RAG Hybride Dense + BM25 avec fusion RRF (Retenu)	Maximale : réconciliation des codes précis et des questions sémantiques	Optimale : combinaison robuste des scores avec dédoublonnage	Retenu : garantit un rappel supérieur à 90% sur la documentation Odoo

Tableau 15. Analyse comparative des modes d'intégration de l'interface utilisateur dans Odoo 17

Mode d'intégration IHM	Expérience utilisateur (UX)	Sécurité et contexte de session	Décision et justification
Iframe web externe	Médiocre : rupture visuelle et redimensionnement malaisé	Complexe : risque d'usurpation de session par cross-frame	Écarté pour des motifs d'ergonomie et de sécurité
Application web indépendante	Faible : contraint l'utilisateur à changer d'onglet de travail	Nécessite une ré-authentification séparée	Écarté car incompatible avec l'usage opérationnel fluide au sein d'Odoo
Composant natif OWL 2.0 (Retenu dans miro_ai)	Excellente : intégration systray et volet rétractable fluide	Natif : transmission directe et vérifiée du contexte utilisateur	Retenu : respect absolu des standards technologiques d'Odoo 17

Chapitre 5. Analyse conceptuelle

5.1. Désignation des rôles de l'équipe SCRUM

La méthodologie SCRUM repose sur trois rôles fondamentaux : le Product Owner, le Scrum Master et l'équipe de développement. Dans le cadre de ce projet, mené en contexte de stage au sein de MiroKE Madagascar, l'équipe est restreinte mais chaque rôle reste clairement identifié, afin de conserver la rigueur du cadre agile tout au long des six mois de réalisation.

Product Owner

Le rôle de Product Owner est assuré par le maître de stage au sein de MiroKE. Ce rôle consiste à exprimer les besoins métier, à définir les priorités fonctionnelles du produit et à valider, à la fin de chaque sprint, que les fonctionnalités livrées répondent effectivement aux attentes opérationnelles. Les choix stratégiques ayant un impact direct sur l'activité de l'entreprise relèvent

également de cette fonction, par exemple la décision d'ouvrir l'accès à l'assistant IA aux clients de MiroKE depuis leur instance Odoo, ou encore la validation des contraintes budgétaires allouées au matériel.

Scrum Master

En l'absence d'un cadre Scrum Master dédié au sein de l'entreprise, ce rôle est intégré aux attributions confiées dans le cadre du stage, en parallèle de la fonction de développement. Cette double attribution, fréquente dans les structures de petite taille où les équipes projet sont réduites, implique que l'application correcte du cadre méthodologique SCRUM soit assurée par la même personne : organisation des sprints, suivi de l'avancement, identification des blocages techniques ou organisationnels, et facilitation des échanges avec le Product Owner lors des revues de sprint. Le contenu fonctionnel du produit ne relève pas de cette fonction, qui garantit en revanche que le processus de travail reste structuré et que les objectifs de chaque itération sont atteignables dans le temps imparti.

Équipe de développement

La fonction d'équipe de développement est également intégrée au périmètre du stage. La conception, le développement et les tests de l'ensemble des composants du projet en relèvent directement : mise en place de l'infrastructure d'inférence locale, construction du pipeline RAG (ingestion, indexation, recherche, génération), développement de l'interface utilisateur et du module d'intégration à Odoo, ainsi que la sécurisation des accès multi-clients. Bien que l'équipe de développement SCRUM soit traditionnellement pluridisciplinaire et composée de plusieurs membres, sa réduction à une seule ressource dans ce contexte de stage ne remet pas en cause la pertinence du cadre méthodologique : les principes de découpage en sprints, de backlog priorisé et de livraison incrémentale restent appliqués à l'identique, à l'échelle d'une ressource unique.

Tableau 16. Synthèse des rôles de l'équipe SCRUM

Rôle SCRUM	Personne assignée	Responsabilités principales
Product Owner	Maître de stage (MiroKE)	Expression des besoins métier, priorisation du backlog, validation des livrables de sprint, arbitrage des décisions stratégiques

Scrum Master	Stagiaire	Organisation des sprints, suivi de l'avancement, facilitation des revues, respect du cadre méthodologique
Équipe de développement	Stagiaire, ingenieur pour l’installation des matériels, maitre de stage	Conception, développement et tests de l'ensemble des composants applicatifs et du module IA

Cette répartition, bien que concentrée sur un nombre réduit d'intervenants, permet de conserver une séparation claire entre la définition des besoins métier (côté Product Owner) et leur mise en œuvre technique (côté équipe de développement), condition nécessaire au bon fonctionnement du cadre SCRUM.

5.2. Étapes de l'élaboration du product backlog

L'élaboration du product backlog s'est appuyée sur une démarche en quatre étapes successives, allant du recueil du besoin brut jusqu'à la formalisation d'une liste d'éléments priorisés et estimables.

Étape 1 — Recueil du besoin

Le recueil du besoin s'est effectué par deux moyens complémentaires. D'une part, des entretiens menés avec le maître de stage ont permis de cadrer les objectifs métier du projet : garantir la confidentialité des données face aux API cloud, réduire le temps de recherche documentaire des équipes, et ouvrir à terme l'accès à l'assistant IA aux clients de MiroKE depuis leur instance Odoo. D'autre part, l'observation du fonctionnement quotidien des équipes support et développement a permis d'identifier les frictions concrètes liées à la dispersion de l'information entre le code source, la documentation technique, les wikis et les tickets de support.

Étape 2 — Formulation des besoins en éléments fonctionnels

Les besoins exprimés lors des entretiens et les frictions observées sur le terrain ont ensuite été reformulés sous forme d'éléments fonctionnels distincts, correspondant chacun à une capacité attendue du système : indexation des sources documentaires, recherche sémantique, génération de

réponses sourcées, interface d'interrogation, isolation des données par client, et sécurisation des accès. Cette reformulation constitue la matière première du product backlog, avant priorisation.

Étape 3 — Rédaction sous forme de user stories

Chaque élément fonctionnel a été traduit en une ou plusieurs user stories, rédigées selon le format standard *"En tant que [rôle], je veux [action], afin de [bénéfice]"*. Ce format permet de relier systématiquement chaque fonctionnalité technique à un bénéfice métier identifiable, condition nécessaire à la validation par le Product Owner.

Étape 4 — Priorisation et estimation

Les user stories ont enfin été priorisées par le Product Owner selon leur valeur métier et leur dépendance technique (une fonctionnalité de recherche sémantique ne peut être développée avant la mise en place de l'indexation, par exemple), puis regroupées en quatre releases correspondant au calendrier de stage de six mois :

Tableau 17. Découpage en releases du projet

Release	Période	Orientation fonctionnelle dominante
Release 1	Mois 1	Étude des besoins, infrastructure, tests de modèles
Release 2	Mois 2-3	Préparation des données internes, pipeline RAG
Release 3	Mois 4-5	Interface utilisateur, intégration aux outils métiers
Release 4	Mois 6	Sécurité, tests, documentation

Cette structuration en releases successives permet une livraison incrémentale de valeur, chaque release produisant un accroissement fonctionnel testable et démontrable au Product Owner, conformément aux principes de la méthodologie SCRUM.

5.3. Product backlog

Le product backlog rassemble l'ensemble des user stories issues du processus décrit en 5.2, classées par release, accompagnées de leur priorité selon la méthode MoSCoW (Must have,

Should have, Could have, Won't have) et de leur estimation en points selon une échelle relative de complexité (1, 2, 3, 5, 8, 13).

Tableau 18. Product Backlog — Release 1 (Infrastructure et modèles)

ID	User story	Priorité	Estimation (pts)
US-01	En tant que Product Owner, je veux qu'une étude comparative des modèles open source (Llama 3, Mistral, Phi-3, DeepSeek) soit réalisée, afin de choisir le modèle le plus adapté au contexte métier	Must	5
US-02	En tant qu'utilisateur final, je veux que le système fonctionne sans connexion internet, afin de garantir la continuité de service en cas de coupure réseau	Must	8
US-03	En tant qu'administrateur système, je veux qu'un environnement d'inférence local (Ollama) soit installé et configuré, afin de pouvoir tester les modèles candidats	Must	5
US-04	En tant que Product Owner, je veux que le matériel nécessaire soit inventorié et budgétisé, afin de valider l'investissement dans l'enveloppe disponible	Must	3
US-05	En tant qu'administrateur système, je veux que les performances de chaque modèle candidat soient mesurées sur le matériel disponible, afin de sélectionner le meilleur compromis qualité/vitesse	Should	5

Tableau 19. Product Backlog — Release 2 (Pipeline RAG et données)

ID	User story	Priorité	Estimation (pts)
US-06	En tant qu'utilisateur, je veux que les documents internes (PDF, wikis, code source) soient indexés dans une base vectorielle, afin que l'assistant puisse les interroger	Must	13

US-07	En tant qu'utilisateur, je veux que mes questions soient associées aux passages pertinents de la documentation, afin d'obtenir une réponse contextualisée	Must	8
US-08	En tant qu'utilisateur, je veux que chaque réponse générée indique sa source exacte (document, section, ou fichier de code), afin de pouvoir vérifier l'information	Must	8
US-09	En tant que développeur support, je veux que le code source Odoo (Python, XML) soit indexé de façon structurée, afin de pouvoir localiser un workflow ou une règle métier précise	Should	8
US-10	En tant qu'administrateur système, je veux que l'ajout d'un nouveau document mette à jour l'index sans nécessiter de réentraînement, afin de garder les réponses à jour en continu	Must	5

Tableau 20. Product Backlog — Release 3 (Interface utilisateur et intégration)

ID	User story	Priorité	Estimation (pts)
US-11	En tant qu'utilisateur interne, je veux disposer d'une interface web simple pour interroger l'assistant, afin de poser mes questions sans passer par la ligne de commande	Must	8
US-12	En tant qu'utilisateur Odoo, je veux accéder à l'assistant directement depuis un module intégré à mon instance Odoo, afin de ne pas changer d'outil	Must	13
US-13	En tant que client de MiroKE, je veux que mes données restent isolées de celles des autres clients, afin de garantir la confidentialité de mes informations métier	Must	8

US-14	En tant qu'utilisateur, je veux que l'historique de mes échanges avec l'assistant ne soit pas partagé avec d'autres utilisateurs, afin de préserver la confidentialité de mes conversations	Should	5
US-15	En tant qu'utilisateur, je veux que l'interface affiche un indicateur de chargement pendant la génération de la réponse, afin de savoir que ma requête est bien prise en compte	Could	2

Tableau 21. Product Backlog — Release 4 (Sécurité, tests et documentation)

ID	User story	Priorité	Estimation (pts)
US-16	En tant qu'administrateur système, je veux que les droits d'accès soient cloisonnés par profil utilisateur, afin d'éviter qu'un employé accède aux données d'un autre client	Must	8
US-17	En tant que Product Owner, je veux que la qualité des réponses sourcées soit mesurée par des métriques objectives, afin de valider la fiabilité du système avant son déploiement en production	Must	5
US-18	En tant qu'administrateur système, je veux que le système soit testé sous charge, afin de m'assurer qu'il supporte plusieurs utilisateurs simultanés	Should	5
US-19	En tant que futur utilisateur, je veux disposer d'une documentation technique et d'un guide utilisateur, afin de pouvoir exploiter le système de façon autonome après le stage	Must	5
US-20	En tant qu'administrateur système, je veux que les accès et requêtes soient journalisés, afin de pouvoir auditer l'utilisation du système en cas d'incident	Could	3

Tableau 22. Synthèse des estimations d'effort du Product Backlog

Release	Nombre de user stories	Total points
Release 1	5	26
Release 2	5	42
Release 3	5	36
Release 4	5	26
Total	20	130

Cette répartition met en évidence une charge plus importante sur la Release 2, correspondant à la construction du pipeline RAG, considérée comme le cœur technique du projet et la principale source de risque du calendrier de stage.

5.4. Sprint backlog

Le calendrier de stage de six mois est découpé en sprints de deux semaines, soit onze sprints au total, répartis sur les quatre releases définies en 5.3. Cette granularité permet un suivi régulier de l'avancement et une adaptation rapide en cas de blocage technique.

5.4.1. Présentation du travail à faire (tableau release/sprint)

Le tableau suivant présente la répartition des sprints au sein de chaque release, ainsi que les user stories qui leur sont rattachées.

Tableau 23. Planification des sprints par release

Release	Sprint	Période	User stories couvertes
Release 1	Sprint 1	Semaines 1-2	US-01, US-04
Release 1	Sprint 2	Semaines 3-4	US-02, US-03, US-05
Release 2	Sprint 3	Semaines 5-6	US-06, US-10
Release 2	Sprint 4	Semaines 7-8	US-07, US-09

Release 2	Sprint 5	Semaines 9-10	US-08
Release 3	Sprint 6	Semaines 11-12	US-11, US-15
Release 3	Sprint 7	Semaines 13-14	US-12
Release 3	Sprint 8	Semaines 15-16	US-13, US-14
Release 4	Sprint 9	Semaines 17-18	US-16, US-20
Release 4	Sprint 10	Semaines 19-20	US-17, US-18
Release 4	Sprint 11	Semaines 21-22	US-19

Le découpage laisse volontairement deux semaines en fin de stage (semaines 23-24) non affectées à un sprint fonctionnel, réservées à la consolidation finale, à la correction des anomalies résiduelles et à la préparation de la soutenance.

5.4.2. Planification du sprint (tableau sprint/tâches)

Chaque user story est décomposée en tâches techniques unitaires lors de la planification de sprint. Le tableau suivant détaille cette décomposition pour les sprints correspondant au cœur technique du projet (Sprints 3 à 5, construction du pipeline RAG), à titre d'illustration de la méthode appliquée à l'ensemble des sprints.

Tableau 24. Sprint Backlog — Sprint 3 (Indexation des documents internes)

Tâche	User story associée	Description
T3.1	US-06	Mettre en place la base vectorielle (ChromaDB) en environnement de développement
T3.2	US-06	Développer le script d'extraction et de découpage (chunking) des documents PDF et wikis
T3.3	US-06	Générer les embeddings des chunks et les insérer dans la base vectorielle
T3.4	US-10	Développer le mécanisme d'ajout incrémental d'un document sans réindexation complète

T3.5	US-10	Tester la mise à jour de l'index après modification d'un document source

Tableau 25. Sprint Backlog — Sprint 4 (Recherche sémantique et code source)

Tâche	User story associée	Description
T4.1	US-07	Implémenter la recherche par similarité vectorielle (top-k) sur une requête utilisateur
T4.2	US-07	Intégrer un mécanisme de recherche hybride combinant similarité sémantique et mots-clés
T4.3	US-09	Adapter le pipeline de chunking pour traiter le code source Python et XML d'Odoo
T4.4	US-09	Tester la récupération de passages de code pertinents sur des requêtes métier types

Tableau 26. Sprint Backlog — Sprint 5 (Génération de réponses sourcées)

Tâche	User story associée	Description
T5.1	US-08	Construire le prompt d'assemblage entre la requête utilisateur et les passages récupérés
T5.2	US-08	Implémenter l'extraction et l'affichage de la source (document, section ou fichier) associée à chaque passage utilisé
T5.3	US-08	Tester la fiabilité du sourcing sur un jeu de questions métier représentatif

Cette même logique de décomposition en tâches techniques est appliquée à l'ensemble des sprints de la planification, garantissant que chaque user story du product backlog est traduite en actions concrètes et estimables avant le début de chaque itération.

5.5. Dictionnaire des données

Le système repose sur deux espaces de stockage distincts : une base relationnelle PostgreSQL, dédiée aux données applicatives propres au module IA (clients, utilisateurs, conversations,

journalisation), et une base vectorielle, dédiée à l'indexation sémantique des documents. La base relationnelle ne duplique aucune donnée métier déjà présente dans Odoo ; elle se contente de référencer les entités Odoo par identifiant, afin de préserver l'intégrité du système d'information existant chez chaque client.

Le dictionnaire ci-dessous présente, par ordre alphabétique des entités, l'ensemble des champs utilisés.

Tableau 27. Dictionnaire des données — Entité Client

Champ	Type	Description
client_id	UUID	Identifiant unique du client de MiroKE (clé primaire)
date_activation	Date	Date d'activation de l'assistant IA pour ce client
nom_entreprise	Varchar(150)	Raison sociale du client
odoo_instance_url	Varchar(255)	URL de l'instance Odoo associée au client
statut	Varchar(20)	État du compte client (actif, suspendu, en test)

Tableau 28. Dictionnaire des données — Entité Conversation

Champ	Type	Description
conversation_id	UUID	Identifiant unique de la conversation (clé primaire)
client_id	UUID	Référence au client concerné (clé étrangère)
date_creation	Timestamp	Date et heure de création de la conversation
utilisateur_id	UUID	Référence à l'utilisateur ayant initié la conversation (clé étrangère)

Tableau 29. Dictionnaire des données — Entité Document_Source

Champ	Type	Description

chemin_origine	Varchar(500)	Emplacement d'origine du document (chemin fichier, URL wiki, chemin code source)
client_id	UUID	Référence au client propriétaire du document (clé étrangère)
date_indexation	Timestamp	Date de la dernière indexation du document
document_id	UUID	Identifiant unique du document source (clé primaire)
type_source	Varchar(30)	Nature du document (PDF, wiki, ticket support, code source)

Tableau 30. Dictionnaire des données — Entité Journal_Acces

Champ	Type	Description
action	Varchar(50)	Type d'action effectuée (requête, consultation source, échec authentification)
date_heure	Timestamp	Horodatage de l'action
journal_id	UUID	Identifiant unique de l'entrée de journal (clé primaire)
resultat	Varchar(20)	Statut de l'action (succès, échec, refusé)
utilisateur_id	UUID	Référence à l'utilisateur ayant effectué l'action (clé étrangère)

Tableau 31. Dictionnaire des données — Entité Message

Champ	Type	Description
conversation_id	UUID	Référence à la conversation parente (clé étrangère)
contenu	Text	Contenu textuel du message (question ou réponse)
date_envoi	Timestamp	Date et heure d'envoi du message
message_id	UUID	Identifiant unique du message (clé primaire)

sources_citees	JSON	Liste structurée des sources utilisées pour générer la réponse (document_id, extrait, score de pertinence)
type_emetteur	Varchar(10)	Origine du message (utilisateur ou assistant)

Tableau 32. Dictionnaire des données — Entité Utilisateur

Champ	Type	Description
client_id	UUID	Référence au client auquel l'utilisateur est rattaché (clé étrangère)
date_creation_compte	Timestamp	Date de création du compte utilisateur
email	Varchar(150)	Adresse électronique de l'utilisateur
odoo_user_id	Integer	Référence à l'identifiant utilisateur correspondant dans Odoo
role	Varchar(30)	Rôle fonctionnel de l'utilisateur (employé MiroKE, employé client, administrateur)
utilisateur_id	UUID	Identifiant unique de l'utilisateur (clé primaire)

La cohérence entre la base relationnelle et la base Odoo de chaque client est assurée par le champ *odoo_user_id*, qui permet de retrouver les droits d'accès réels définis dans Odoo (res.groups, ir.rule) sans dupliquer ces informations dans la base applicative du module IA.

5.6. Règles de gestion

Les règles de gestion formalisent les contraintes métier auxquelles le système doit se conformer, indépendamment des choix techniques retenus pour son implémentation. Elles sont issues directement des objectifs et des contraintes identifiés au chapitre 4, et conditionnent la conception des traitements présentés dans la suite de ce chapitre.

RG-01 — Isolation des données par client

Les données d'un client de MiroKE ne peuvent en aucun cas être accessibles, directement ou indirectement, par un autre client. Chaque requête formulée par un utilisateur est restreinte à la collection documentaire correspondant à son client_id.

RG-02 — Traçabilité obligatoire des réponses

Toute réponse générée par l'assistant doit être accompagnée d'au moins une source identifiable (document, section ou fichier de code). Une réponse pour laquelle aucune source pertinente n'a pu être retrouvée doit l'indiquer explicitement à l'utilisateur plutôt que de générer une réponse non sourcée.

RG-03 — Aucune transmission de données hors infrastructure locale

Aucune donnée métier, document interne ou requête utilisateur ne peut être transmise à un service externe situé hors de l'infrastructure locale de MiroKE ou de ses clients. Cette règle exclut l'usage de toute API de modèle de langage hébergée sur un cloud public dans le traitement des requêtes.

RG-04 — Cloisonnement des droits d'accès

Un utilisateur ne peut consulter que les documents et informations pour lesquels son profil (défini dans Odoo via res.groups et ir.rule) lui accorde un accès. Le module IA ne peut pas accorder un accès plus large que celui défini par les droits Odoo existants.

RG-05 — Mise à jour continue de l'index documentaire

Toute modification, ajout ou suppression d'un document source doit être reflétée dans l'index de recherche dans un délai raisonnable, sans nécessiter de réentraînement du modèle de langage utilisé.

RG-06 — Continuité de service hors connexion internet

Le système doit demeurer pleinement opérationnel en cas d'interruption de la connectivité internet, à l'exception des fonctionnalités explicitement dépendantes d'un service externe.

RG-07 — Journalisation des accès

Toute requête adressée à l'assistant, ainsi que toute consultation de source associée à une réponse, doit être journalisée avec l'identifiant de l'utilisateur concerné, à des fins d'audit et de sécurité.

RG-08 — Validation des livrables par le Product Owner

Aucune fonctionnalité développée au cours d'un sprint n'est considérée comme acquise au product backlog avant validation explicite par le Product Owner lors de la revue de sprint correspondante.

Ces règles de gestion serviront de critères de validation lors des tests fonctionnels présentés au chapitre 9, chacune d'elles devant pouvoir être vérifiée de façon objective sur le système livré.

5.7. Diagramme de cas d'utilisation global

Le diagramme de cas d'utilisation global présente une vue d'ensemble des interactions possibles entre les différents acteurs du système et les fonctionnalités offertes par l'assistant IA documentaire. Quatre acteurs sont identifiés :

- l'**utilisateur interne**, employé de MiroKE (support ou développement), qui interroge l'assistant pour retrouver une information opérationnelle ou technique ;
- l'**utilisateur client**, employé d'un client de MiroKE accédant à l'assistant depuis son instance Odoo ;
- l'**administrateur système**, responsable de la configuration, de l'indexation des documents et de la supervision du système ;
- le **système Odoo**, acteur secondaire qui fournit les informations de droits d'accès utilisées pour l'authentification.

La figure suivante illustre l'ensemble des cas d'utilisation et leurs relations.

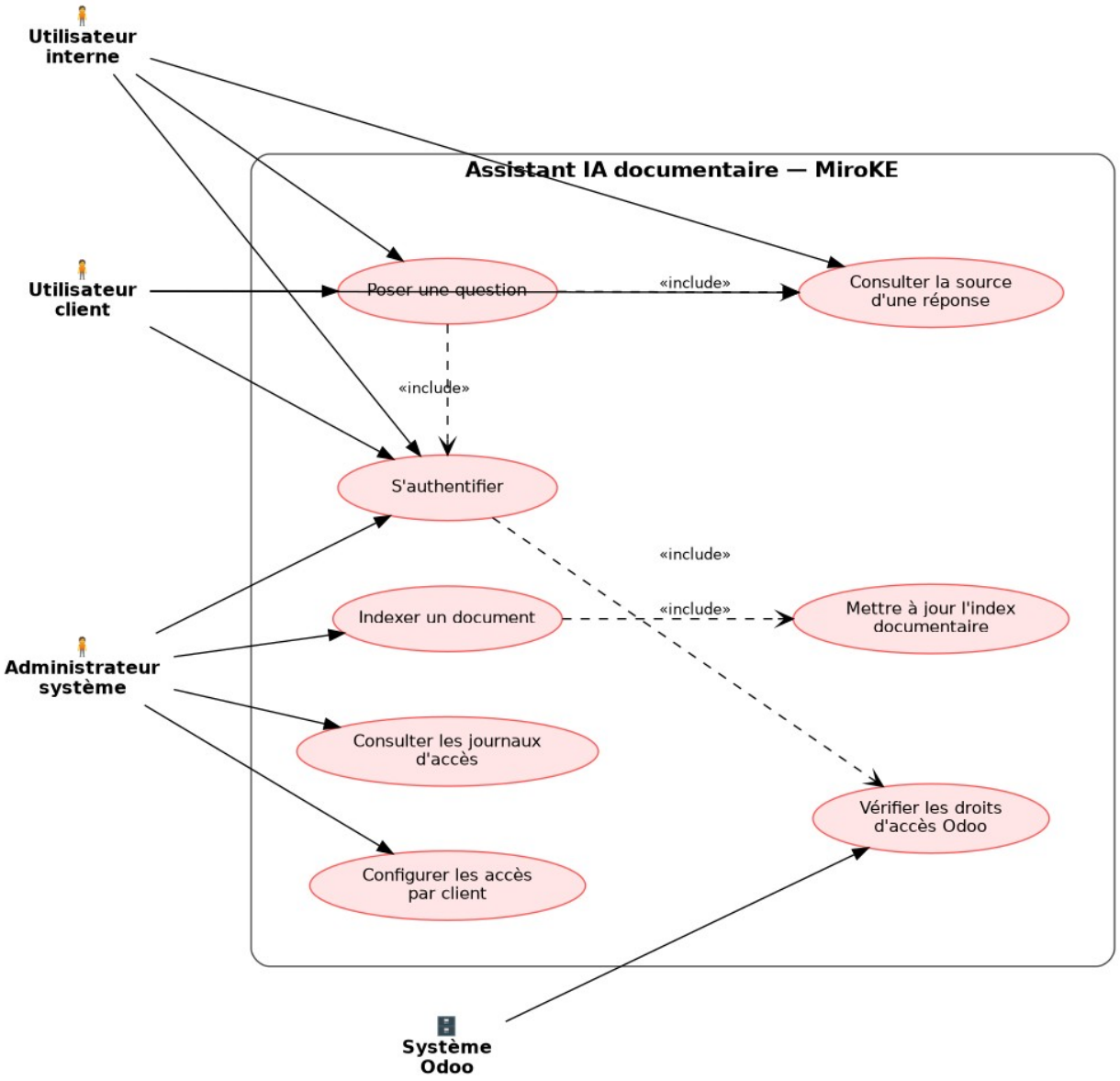


Figure 5.1 — Diagramme de cas d'utilisation global du système

Le cas d'utilisation central, *Poser une question*, inclut systématiquement *Consulter la source d'une réponse*, conformément à la règle de gestion RG-02 selon laquelle aucune réponse ne peut être générée sans indication de sa provenance. Il inclut également *S'authentifier*, qui dépend à son tour de la vérification des droits d'accès définis dans Odoo (*Vérifier les droits d'accès Odoo*), garantissant ainsi le respect du cloisonnement défini par la règle RG-04.

Du côté de l'administration, le cas *Indexer un document* inclut *Mettre à jour l'index documentaire*, ce qui correspond au mécanisme de mise à jour continue de l'index décrit en RG-

05. Les cas *Configurer les accès par client* et *Consulter les journaux d'accès* complètent les responsabilités de l'administrateur système en matière de cloisonnement multi-tenant et d'audit.

Ce diagramme constitue la base sur laquelle s'appuient le modèle du domaine (5.8) et les diagrammes de classes de conception (5.9 et 5.10), chaque cas d'utilisation identifié ici trouvant sa traduction en classes et en méthodes dans les sections suivantes.

5.8. Modèle du domaine

Le modèle du domaine représente les concepts métier manipulés par le système ainsi que les relations qui les unissent, indépendamment de toute considération technique d'implémentation. Il constitue le pont entre les cas d'utilisation identifiés en 5.7 et les diagrammes de classes de conception présentés en 5.9 et 5.10.

Neuf concepts métier sont identifiés :

- **Client** représente une entreprise utilisatrice de MiroKE disposant de sa propre instance Odoo et de sa propre base documentaire isolée ;
- **Utilisateur** représente toute personne autorisée à interroger l'assistant, qu'elle soit employée de MiroKE ou d'un client ;
- **Conversation** regroupe les échanges successifs entre un utilisateur et l'assistant ;
- **Message** correspond à un élément unitaire d'échange, qu'il s'agisse d'une question posée par l'utilisateur ou d'une réponse générée par l'assistant ;
- **Source** matérialise le lien entre une réponse générée et le passage documentaire ayant servi à sa construction, conformément à l'exigence de traçabilité posée en RG-02 ;
- **DocumentSource** représente un document brut faisant partie de la base de connaissance d'un client (PDF, page de wiki, fichier de code source) ;
- **IndexDocumentaire** représente l'ensemble structuré et interrogeable des documents indexés pour un client donné ;
- **JournalAcces** trace chaque action effectuée par un utilisateur, conformément à la règle de gestion RG-07 ;
- **DroitAccesOdoo** représente la projection, au sein du domaine, des droits d'accès définis nativement dans Odoo, garantissant le respect du cloisonnement décrit en RG-04.

La figure suivante illustre l'ensemble de ces concepts et leurs cardinalités.

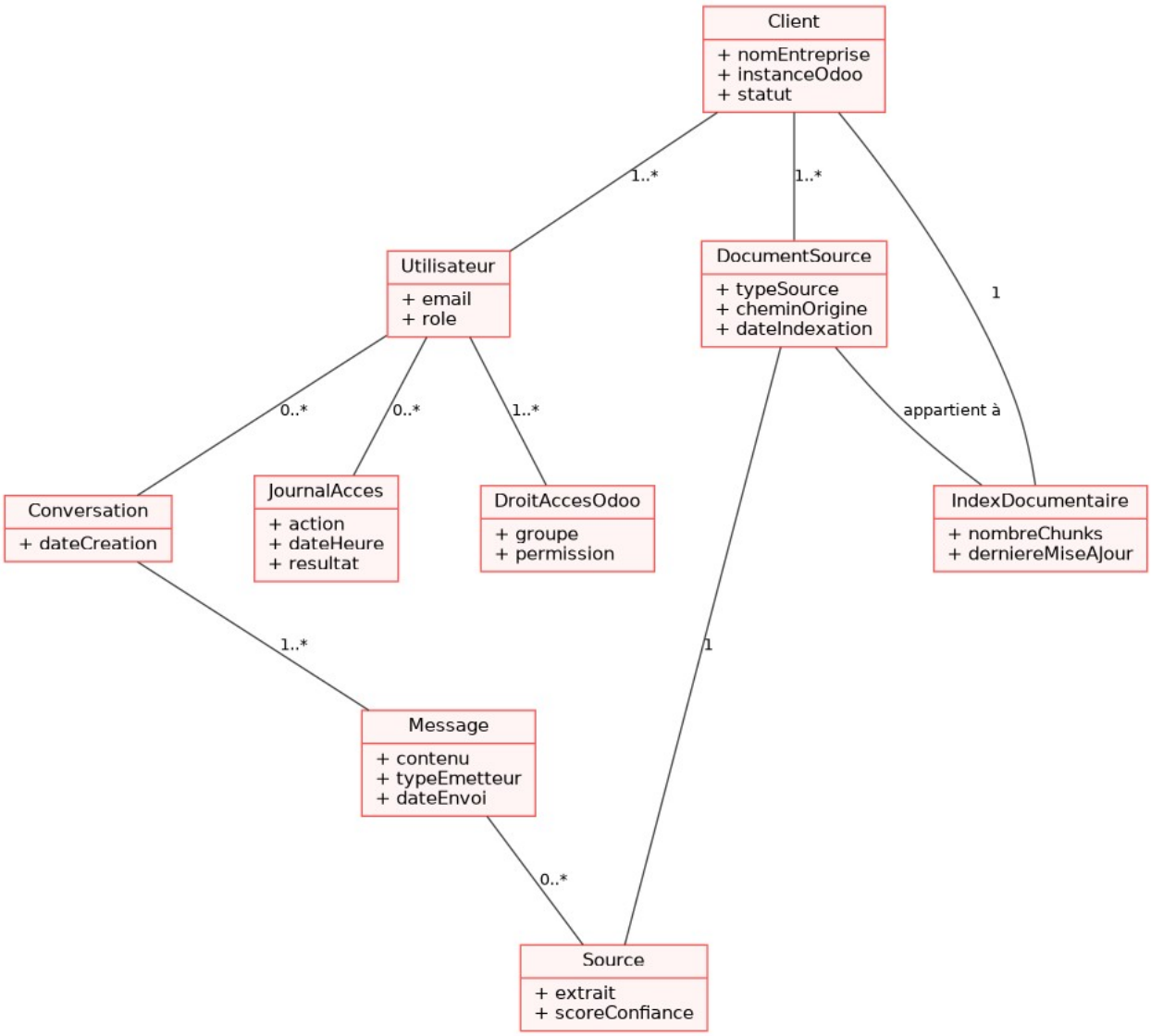


Figure 5.2 — Modèle du domaine du système

Le modèle met en évidence deux ramifications distinctes issues du concept central **Client** : une ramification orientée vers la gestion des utilisateurs et de leurs droits (Utilisateur, JournalAcces, DroitAccesOdoo), et une ramification orientée vers la base de connaissance (DocumentSource, IndexDocumentaire). Ces deux ramifications se rejoignent au niveau du concept **Source**, qui relie chaque message généré par l'assistant à un document précis de la base de connaissance, assurant ainsi la cohérence entre les exigences de personnalisation métier et de traçabilité posées dès le chapitre 4.

Ce modèle ne préjuge d'aucun choix d'implémentation technique : la correspondance entre **DocumentSource** et **IndexDocumentaire** recouvre, dans l'architecture retenue, à la fois la base relationnelle PostgreSQL et la base vectorielle dédiée à la recherche sémantique, dont la structure spécifique est détaillée en 6.2.

5.9. Diagramme de classe de conception pour chaque sprint

Conformément au découpage en sprints présenté en 5.4, chaque itération fait l'objet d'une conception détaillée au niveau des classes. Seuls les sprints introduisant de nouvelles classes de conception sont illustrés par un diagramme dédié dans cette section ; les sprints restants (2, 6, 8, 10 et 11) consistent en l'enrichissement de méthodes sur des classes déjà posées lors d'un sprint antérieur, sans modification de la structure de classes, et ne sont donc pas représentés séparément.

Sprint 1 — Infrastructure et configuration

Ce sprint pose les fondations de l'environnement d'inférence local. La classe *GestionnaireInference* orchestre le cycle de vie du service de modèle de langage, en s'appuyant sur une *ConfigSysteme* pour les paramètres d'exécution et sur une instance de *ModeleIA* représentant le modèle effectivement chargé (Mistral 7B ou Llama 3 8B selon les résultats des tests comparatifs menés durant ce sprint).

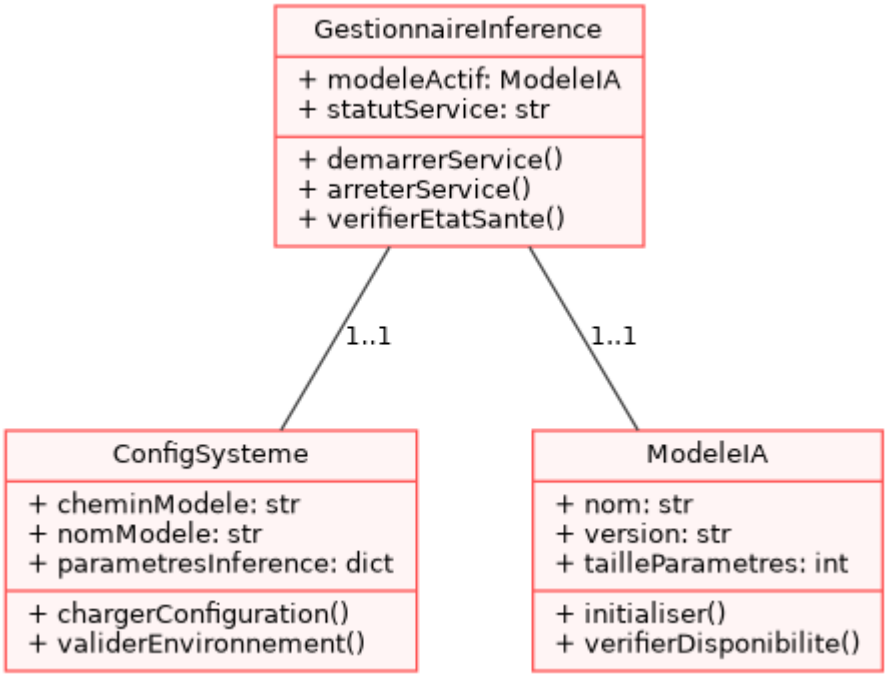


Figure 5.3 — Diagramme de classes de conception, Sprint 1

Sprint 3 — Indexation des documents

Ce sprint introduit le cœur du pipeline d'indexation. Un *DocumentSource* est découpé en plusieurs instances de *ChunkTexte*, chacune associée à un *Embedding* vectoriel. L'*IndexVectoriel* assure la gestion de l'ajout, de la réindexation et de la suppression des chunks, répondant directement à l'exigence de mise à jour continue posée en RG-05.

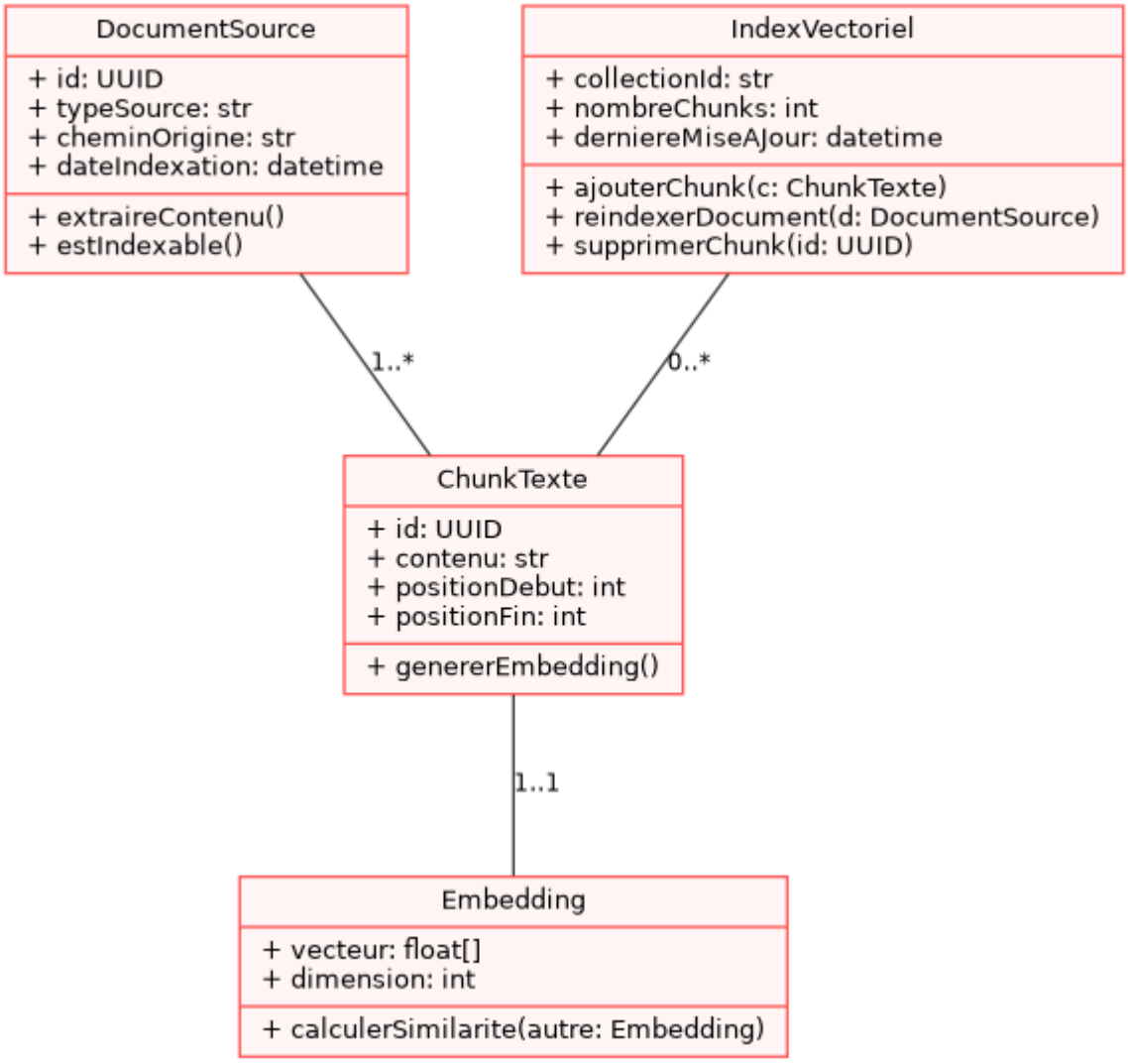


Figure 5.4 — Diagramme de classes de conception, Sprint 3

Sprint 4 — Recherche sémantique et indexation du code source

Ce sprint introduit le moteur de recherche proprement dit. Une *RequeteRecherche* est traitée par le *MoteurRechercheHybride*, qui combine une recherche par similarité vectorielle et une recherche par mots-clés avant de produire une liste de *ResultatRecherche* triée par pertinence. L'*AnalyseurCodeSource* est introduit spécifiquement pour traiter les particularités du code source Odoo (Python, XML), nécessaire à la satisfaction de la user story US-09.

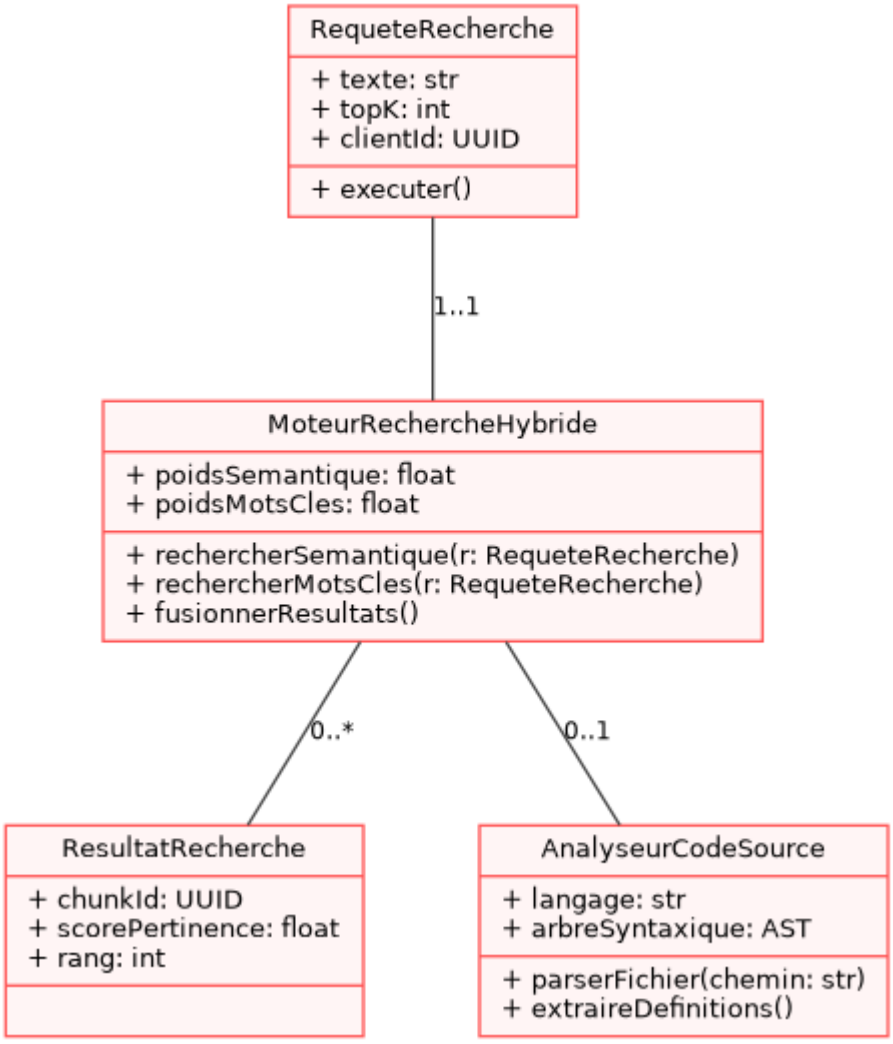


Figure 5.5 — Diagramme de classes de conception, Sprint 4

Sprint 5 — Génération de réponses sourcées

Ce sprint clôt la chaîne de traitement du pipeline RAG. L'*AssembleurPrompt* construit la requête finale destinée au modèle à partir des résultats de recherche, tandis que le *GenerateurReponse* produit le *Message* de réponse et lui associe une ou plusieurs instances de *Source*, garantissant la traçabilité exigée par la règle de gestion RG-02.

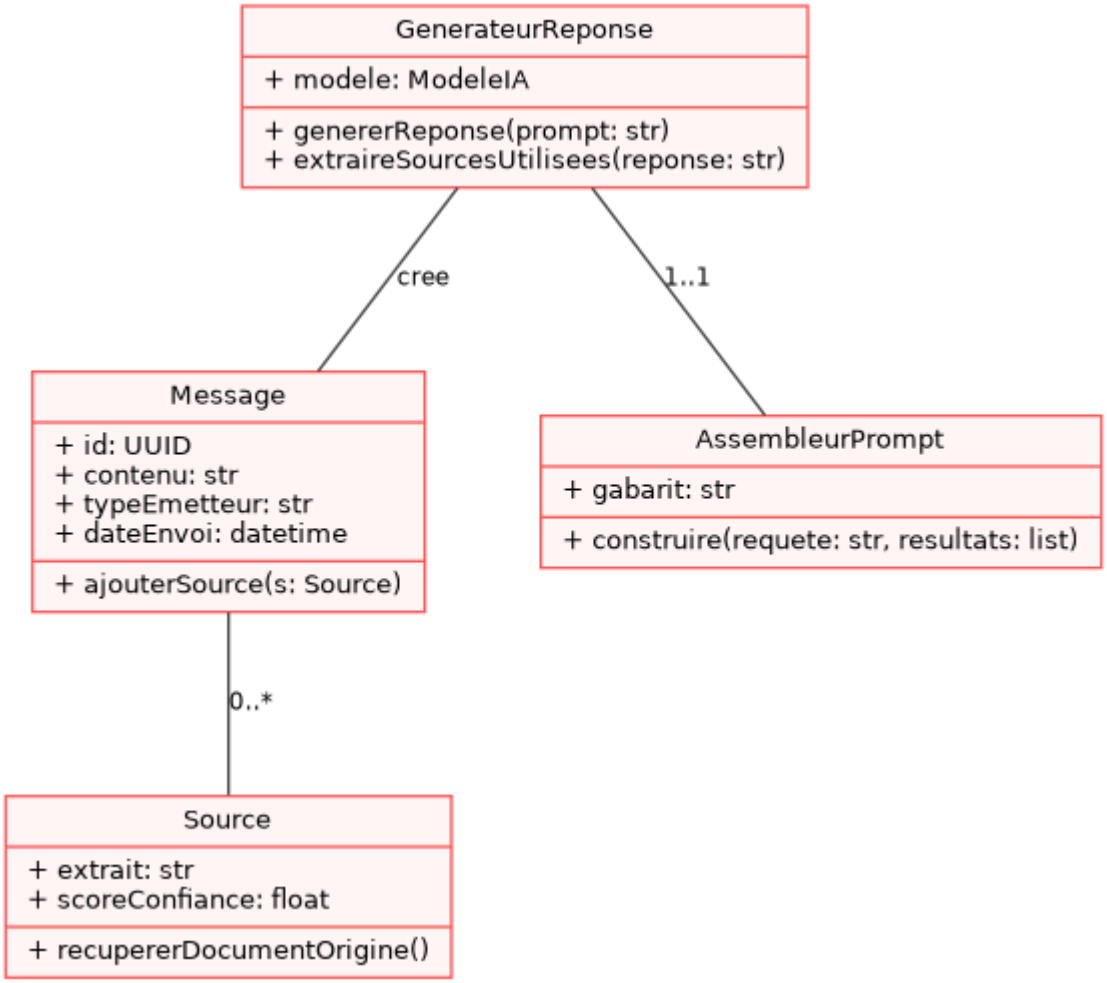


Figure 5.6 — Diagramme de classes de conception, Sprint 5

Sprint 7 — Module d'intégration Odoo

Ce sprint introduit les classes nécessaires à l'accès à l'assistant depuis l'instance Odoo d'un client. Le *ModuleOdoo* s'appuie sur un *ConnecteurAPI* pour communiquer avec le backend du pipeline RAG, et sur une *SessionOdoo* pour authentifier l'utilisateur courant à partir de son contexte Odoo natif.

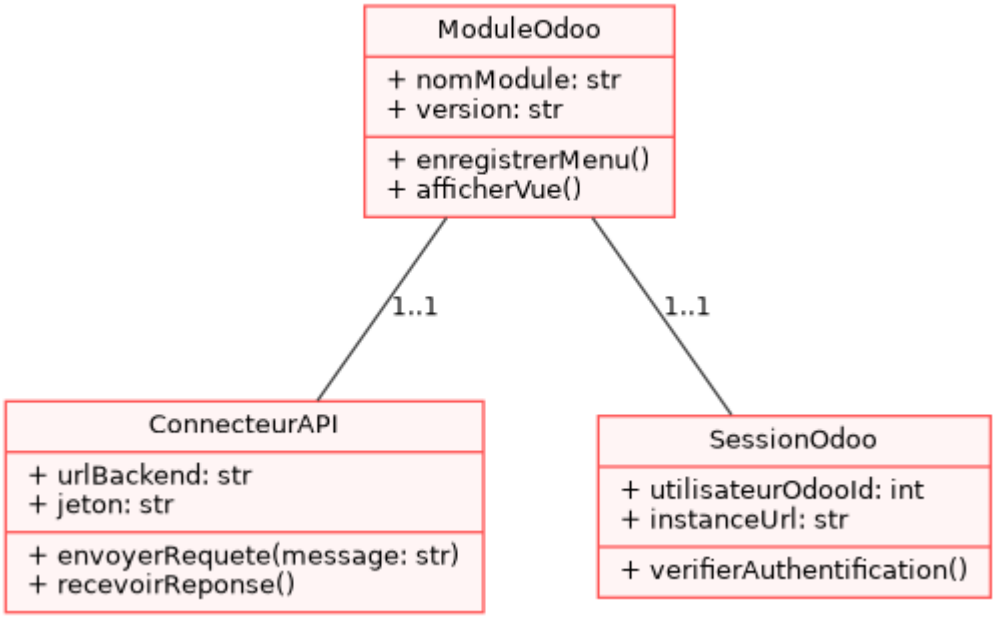


Figure 5.7 — Diagramme de classes de conception, Sprint 7

Sprint 9 — Sécurité et cloisonnement multi-tenant

Ce sprint introduit les classes garantissant le respect des règles de gestion RG-01 et RG-04. Le *GestionnaireAcces* vérifie, pour chaque *Utilisateur*, les permissions associées à son *DroitAccesOdoo* avant d'autoriser l'accès à une ressource, et produit une entrée de *JournalAcces* à chaque vérification, conformément à RG-07.

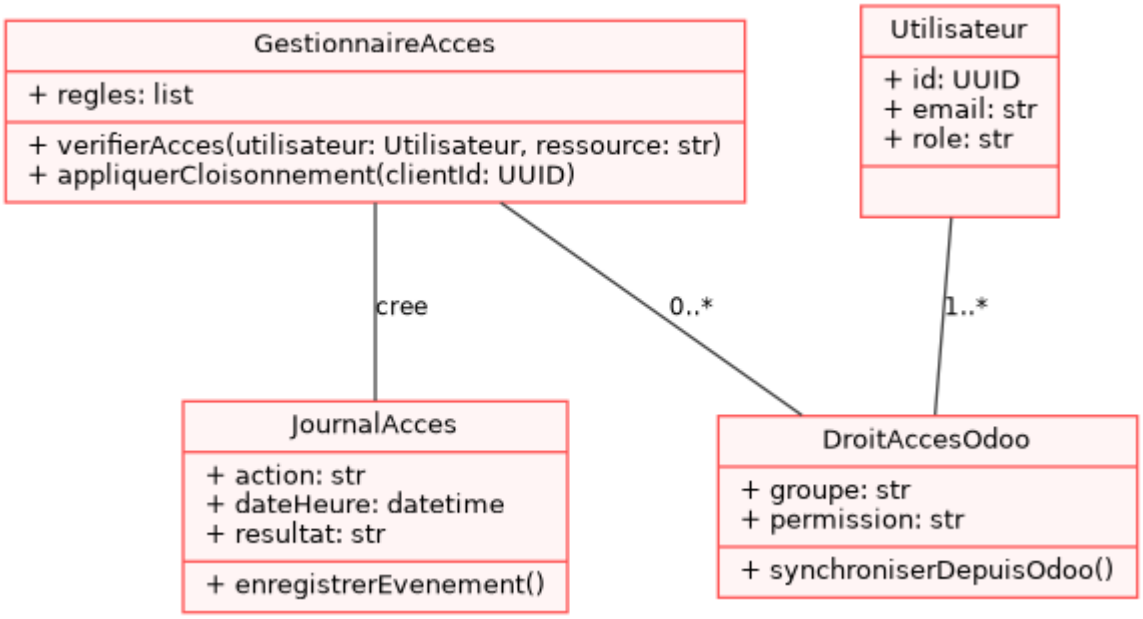


Figure 5.8 — Diagramme de classes de conception, Sprint 9

L'ensemble de ces classes, introduites progressivement au fil des sprints, sont consolidées dans le diagramme de classes de conception globale présenté en 5.10.

5.10. Diagramme de classe de conception globale

Le diagramme de classes de conception globale consolide l'ensemble des classes introduites progressivement au fil des sprints présentés en 5.9, organisées en cinq couches architecturales cohérentes avec la structure du système décrite au chapitre 4 : Interface & Intégration, Sécurité & Multi-tenant, Recherche & Génération, Ingestion & Indexation, et Infrastructure.

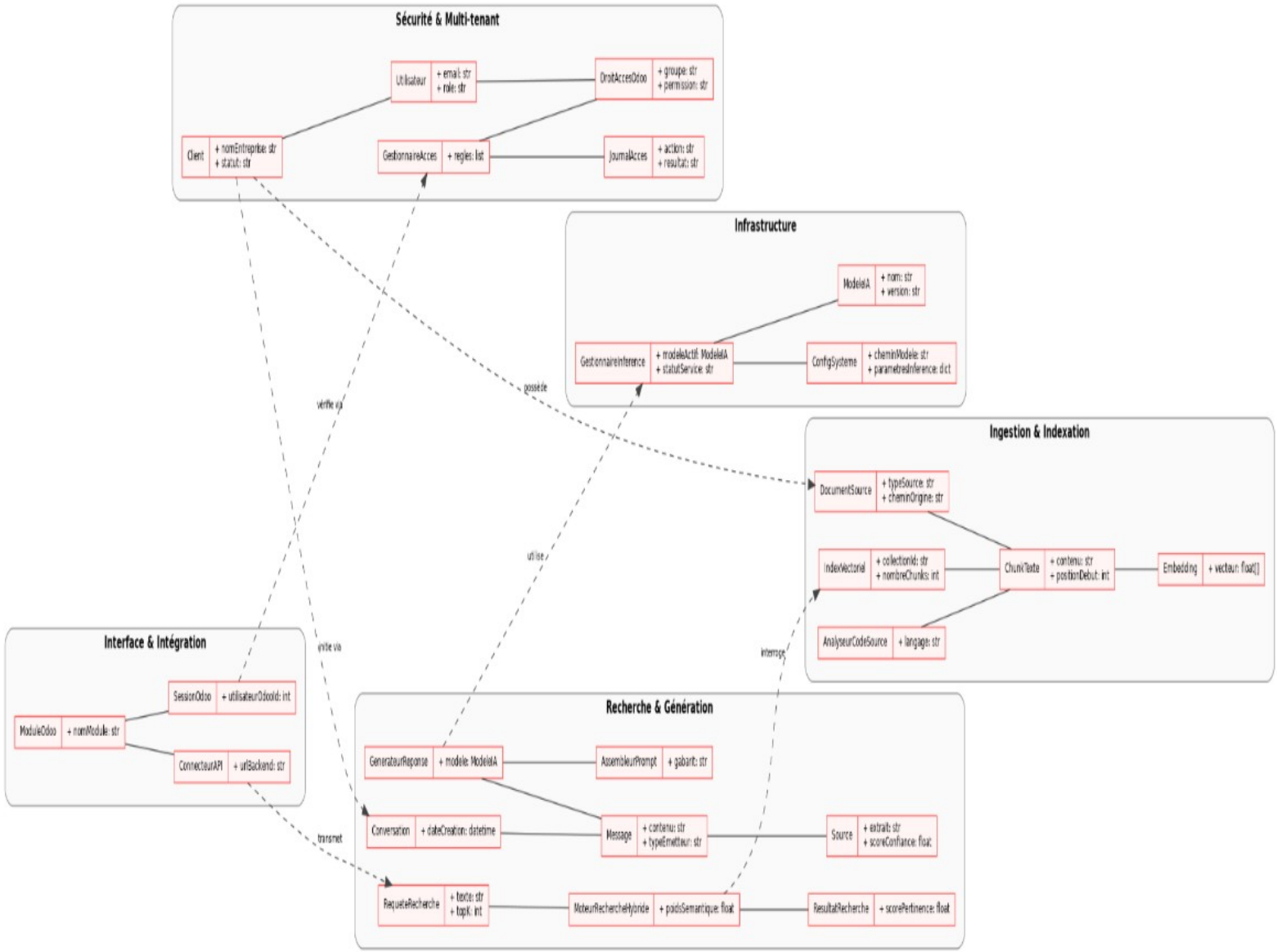

Sécurité & Multi-tenant
Utilisateur
+ email: str
+ role: str
DroitAccesOdoo
+ groupe: str
+ permission: str
Client
+ nomEntreprise: str
+ statut: str
GestionnaireAcces
+ regles: list
JournalAcces
+ action: str
+ resultat: str
Infrastructure
ModeleIA
+ nom: str
+ version: str
GestionnaireInference
+ modeleActif: ModeleIA
+ statutService: str
ConfigSysteme
+ cheminModele: str
+ parametresInference: dict
Ingestion & Indexation
DocumentSource
+ typeSource: str
+ cheminOrigine: str
IndexVectoriel
+ collectionId: str
+ nombreChunks: int
ChunkTexte
+ contenu: str
+ positionDebut: int
Embedding
+ vecteur: float[]
AnalyseurCodeSource
+ langage: str
Interface & Intégration
SessionOdoo
+ utilisateurOdooId: int
ModuleOdoo
+ nomModule: str
ConnecteurAPI
+ urlBackend: str
Recherche & Génération
GenerateurReponse
+ modele: ModeleIA
AssembleurPrompt
+ garantir: str
Conversation
+ dateCreation: datetime
Message
+ contenu: str
+ typeEmetteur: str
Source
+ extrait: str
+ scoreConfiance: float
RequeteRecherche
+ texte: str
+ topK: int
MoteurRechercheHybride
+ poidsSemantique: float
ResultatRecherche
+ scorePertinence: float
possède
vérifie via
utilise
invite via
interroge
transmet

Figure 5.9 — Diagramme de classes de conception globale du système

Couche Interface & Intégration

Cette couche regroupe les classes responsables de l'accès au système depuis l'instance Odoo d'un client. Le *ModuleOdoo* s'appuie sur un *ConnecteurAPI* pour transmettre les requêtes au backend, et sur une *SessionOdoo* pour identifier l'utilisateur courant.

Couche Sécurité & Multi-tenant

Cette couche porte la responsabilité du cloisonnement entre clients et du contrôle d'accès. Le *Client* regroupe ses *Utilisateur*, dont les permissions sont vérifiées par le *GestionnaireAcces* à partir des *DroitAccesOdoo* synchronisés depuis Odoo, chaque vérification donnant lieu à une entrée de *JournalAcces*.

Couche Recherche & Génération

Cette couche constitue le cœur fonctionnel du pipeline RAG. Une *Conversation* regroupe des *Message*, chacun pouvant être associé à une ou plusieurs *Source*. Le traitement d'une *RequeteRecherche* par le *MoteurRechercheHybride* produit des *ResultatRecherche*, ensuite assemblés par l'*AssembleurPrompt* avant d'être transmis au *GenerateurReponse*.

Couche Ingestion & Indexation

Cette couche assure la transformation des documents bruts en représentations exploitables par le moteur de recherche. Un *DocumentSource* est décomposé en *ChunkTexte*, chacun associé à un *Embedding* vectoriel, l'ensemble étant maintenu par l'*IndexVectoriel*. L'*AnalyseurCodeSource* complète cette couche pour le traitement spécifique du code source Odoo.

Couche Infrastructure

Cette couche, la plus stable du système, gère le cycle de vie du modèle de langage local. Le *GestionnaireInference* s'appuie sur une *ConfigSysteme* pour initialiser et superviser le *ModeleIA* effectivement chargé en mémoire.

Dépendances inter-couches

Six relations de dépendance traversent les couches et matérialisent le flux de traitement global d'une requête : le *ConnecteurAPI* transmet la requête au moteur de recherche, le *Client* possède ses propres documents et conversations, le moteur de recherche interroge l'index vectoriel correspondant au client courant, le générateur de réponse s'appuie sur le gestionnaire d'inférence, et la session Odoo passe par le gestionnaire d'accès avant toute opération. Cette organisation en couches faiblement couplées, où chaque couche n'expose que les classes nécessaires aux couches voisines, facilite l'évolution indépendante de chaque composant et la maintenance du système après la fin du stage.

5.11. Diagramme de paquetage

(Numéroté 5.9 dans le canevas d'origine ; renuméroté 5.11 dans ce mémoire afin d'éviter la duplication avec la section 5.9 « Diagramme de classe de conception pour chaque sprint » et la section 5.10 « Diagramme de classe de conception globale », déjà attribuées ci-dessus.)

Le diagramme de paquetage organise le système en cinq modules logiques, correspondant aux couches architecturales déjà identifiées dans le diagramme de classes de conception globale (5.10). Cette vue met en évidence les dépendances de compilation et d'exécution entre modules, sans entrer dans le détail des classes qui les composent.

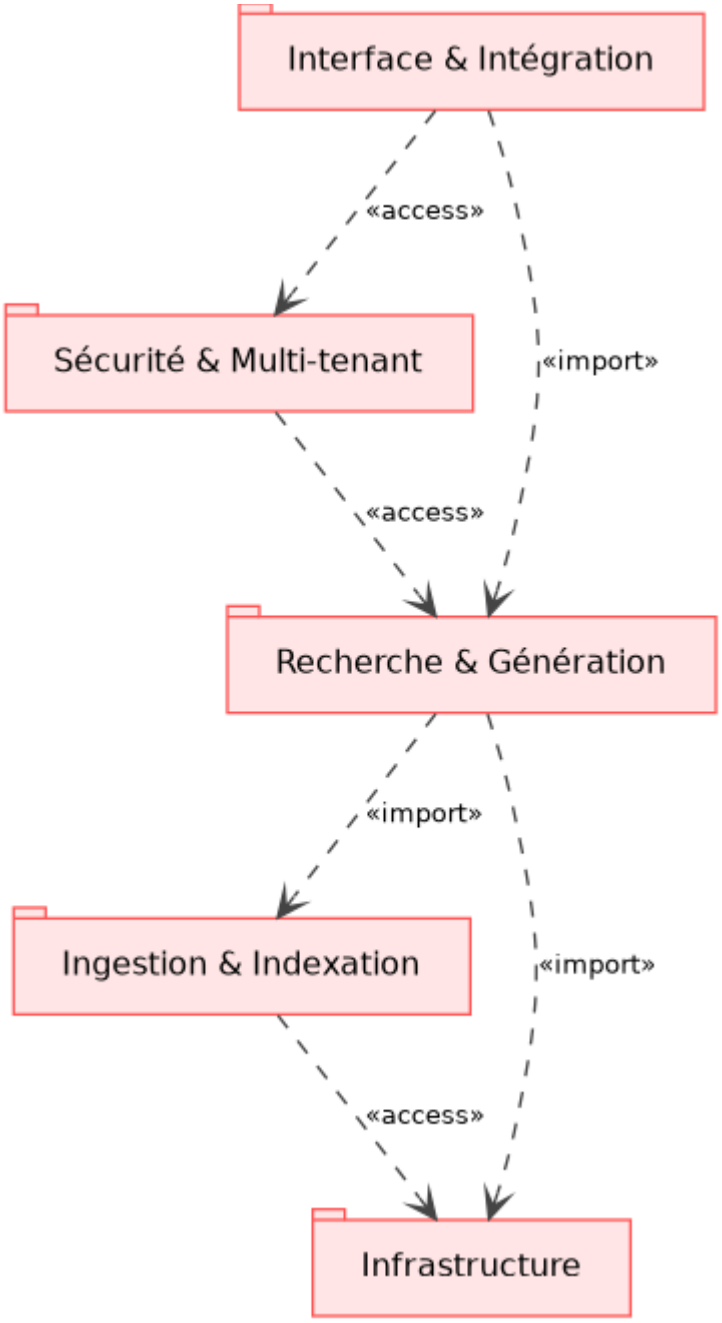


Figure 5.10 — Diagramme de paquetage du système

Le paquetage **Interface & Intégration** dépend du paquetage **Sécurité & Multi-tenant** pour vérifier les droits d'un utilisateur avant de transmettre une requête, ainsi que du paquetage **Recherche & Génération** pour l'exécution proprement dite de la requête. Le paquetage **Sécurité & Multi-tenant** accède au paquetage **Recherche & Génération** afin de restreindre les résultats de recherche au périmètre documentaire autorisé pour l'utilisateur courant.

Le paquetage **Recherche & Génération** dépend à son tour du paquetage **Ingestion & Indexation**, dont il consomme l'index vectoriel pour la recherche sémantique, ainsi que du paquetage **Infrastructure**, dont il sollicite le service d'inférence pour la génération de la réponse. Enfin, le paquetage **Ingestion & Indexation** accède au paquetage **Infrastructure** pour le calcul des embeddings lors de l'indexation d'un nouveau document.

Cette organisation en dépendances strictement descendantes, sans cycle entre paquetages, garantit qu'aucun module de bas niveau (Infrastructure, Ingestion & Indexation) ne dépend d'un module de plus haut niveau. Une telle architecture facilite les tests unitaires de chaque paquetage de façon isolée et limite la propagation des modifications futures à un nombre restreint de modules.

5.12. Diagramme de déploiement

Le diagramme de déploiement décrit la répartition physique des composants logiciels du système sur les différents équipements impliqués, conformément aux exigences de confidentialité et de fonctionnement local posées dès le chapitre 3.

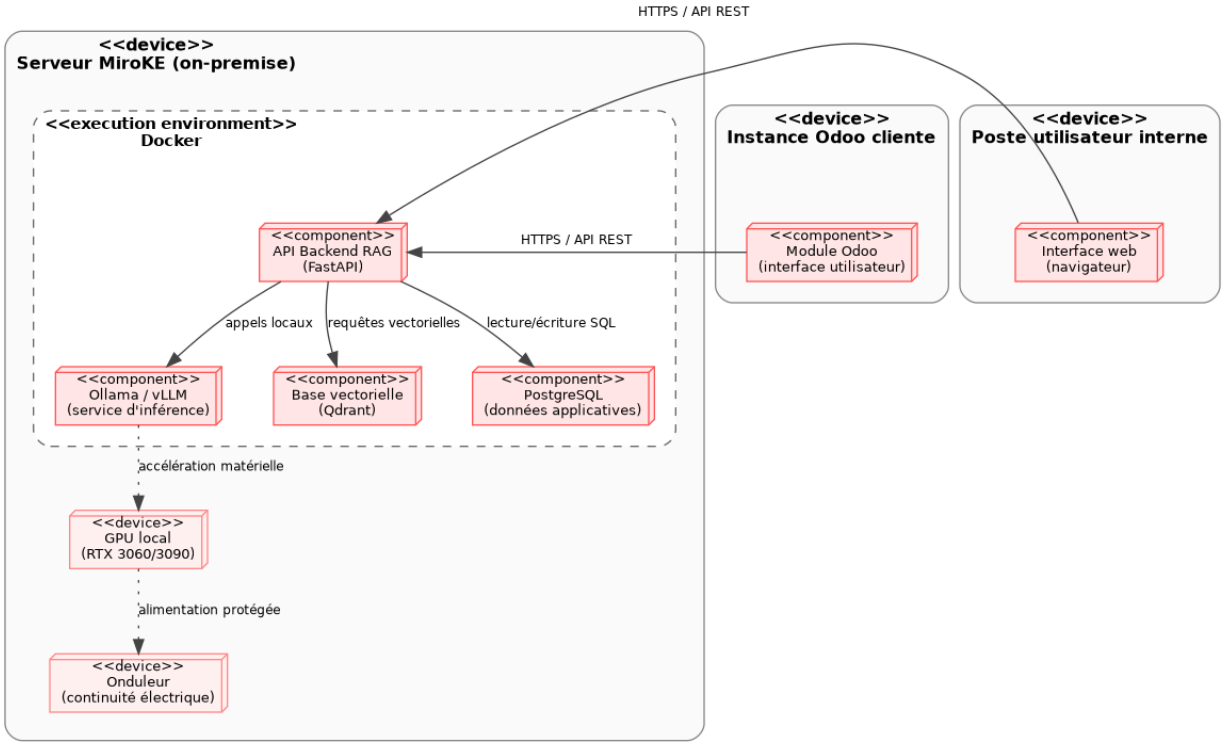


Figure 5.11 — Diagramme de déploiement du système

Serveur MiroKE (on-premise)

L'ensemble des composants logiciels sensibles est hébergé sur un serveur physique situé dans les locaux de MiroKE, conformément à la règle de gestion RG-03 interdisant toute transmission de données vers un service externe. Les composants applicatifs (service d'inférence, API backend, base vectorielle, base relationnelle) sont conteneurisés sous Docker, ce qui permet d'isoler chaque service et de faciliter son redéploiement. Le serveur s'appuie sur un GPU local pour l'accélération du calcul d'inférence, et sur un onduleur garantissant la continuité de l'alimentation électrique face aux coupures de courant fréquentes évoquées au chapitre 4.

API Backend RAG

Ce composant central, développé avec FastAPI, orchestre les échanges entre les trois autres services : il transmet les requêtes de génération au service d'inférence (Ollama en développement, vLLM en production), interroge la base vectorielle pour la recherche sémantique, et lit ou écrit dans la base PostgreSQL pour la gestion des données applicatives (utilisateurs, conversations, journaux d'accès).

Instance Odoo cliente

Chaque client de MiroKE dispose d'un module Odoo installé sur sa propre instance, qui communique avec l'API backend du serveur MiroKE via une connexion HTTPS sécurisée. Ce déploiement préserve le principe de cloisonnement multi-tenant : le module Odoo ne transmet que les requêtes de son client, identifié par son client_id, sans jamais accéder directement à l'infrastructure serveur.

Poste utilisateur interne

Les employés de MiroKE accèdent à l'assistant par une interface web standard, exécutée dans un navigateur, qui communique avec la même API backend selon le même protocole HTTPS.

Cette architecture de déploiement centralise l'ensemble du traitement sensible sur une infrastructure unique et maîtrisée par MiroKE, tandis que les points d'accès distants (instances Odoo clientes, postes internes) ne font que transmettre des requêtes et afficher des résultats, sans jamais héberger de donnée métier ni de modèle de langage.

Chapitre 6. Intégration IA dans la conception

L'introduction de capacités d'intelligence artificielle au sein d'un Progiciel de Gestion Intégré tel qu'Odoo 17 ne se limite pas à l'appel distant d'un modèle de langage. Elle exige une conception d'ingénierie rigoureuse afin d'articuler harmonieusement les flux de données hétérogènes, de garantir la souveraineté des traitements et de maintenir le contrôle d'accès strict propre au système d'information de l'entreprise. Ce chapitre formalise l'intégration de l'IA dans le système : la définition détaillée des modules composants l'assistant MIRO AI, la caractérisation des flux de données nécessaires, la justification technique des modèles retenus (embedding dense et LLM de génération) et enfin les modalités d'intégration par microservice asynchrone.

6.1. Définition du module IA

Le système MIRO AI est conçu selon une architecture modulaire et découplée articulée autour de deux composants logiciels fondamentaux en interaction continue :

- **Le module client Odoo 17 (« miro_ai ») :** Développé nativement en JavaScript OWL 2.0 (Odoo Web Library) et Python Odoo, ce composant s'insère directement dans le client web d'Odoo. Il fournit une interface conversationnelle ergonomique accessible depuis la barre supérieure (systray), supporte le rendu dynamique du Markdown, gère l'historique des échanges et affiche les badges de confiance et les sources citées. Côté serveur Odoo, un contrôleur asynchrone sécurisé relaie les requêtes des utilisateurs vers le moteur IA en y associant l'identité stricte (uid), la société active (company_id) et les groupes de privilèges, tout en assurant la traçabilité complète dans le journal d'audit miro.ai.log.
- **Le serveur middleware d'orchestration IA (FastAPI & LangGraph) :** Service backend autonome déployé sur serveur sécurisé privé. Il héberge un graphe décisionnel LangGraph chargé de classer la question, d'orienter la requête vers le canal approprié (recherche documentaire ou consultation système ERP) et de piloter les moteurs d'exécution spécialisés.

Au sein du backend d'orchestration, deux moteurs spécialisés opèrent selon la nature de la sollicitation :

- **Le moteur RAG documentaire hybride :** Dédié à la recherche dans les connaissances d'entreprise non structurées (manuels de procédures, guides techniques, wikis internes, politiques de gestion). Il combine une recherche sémantique dense (embeddings BGE-M3) et une recherche lexicale exacte (Rank-BM25), fusionnées par l'algorithme Reciprocal Rank Fusion (RRF). Un module d'assemblage et de vérification anti-hallucination contraint le LLM à étayer chaque affirmation par une citation explicite des passages documentaires récupérés.
- **L'agent d'accès et de lecture ORM en temps réel :** Spécifiquement conçu pour l'interrogation des données transactionnelles et de gestion d'Odoo. Cet agent exploite un ensemble d'outils typés en lecture seule (inspection du schéma, comptage, listage et agrégation d'indicateurs). Il exécute impérativement chaque requête sous le contexte de

sécurité authentifié du demandeur (uid), respectant scrupuleusement les droits de lecture (ir.model.access) et les filtres de cloisonnement multi-sociétés (ir.rule) sans jamais recourir à sudo() ni à des requêtes SQL brutes non contrôlées.

6.2. Données nécessaires au système

Le bon fonctionnement de MIRO AI repose sur l'exploitation coordonnée de trois catégories de données distinctes mais complémentaires :

• **Données documentaires non structurées (Corpus de connaissances) :** Fichiers PDF, pages de wikis, documentations techniques Markdown, comptes rendus opérationnels et tickets de support archivés. Ces documents font l'objet d'un pré-traitement rigoureux : extraction textuelle, nettoyage des caractères parasites, segmentation sémantique en blocs homogènes (chunks de 512 unités avec un recouvrement de 50 unités pour préserver la continuité du contexte) et vectorisation.

• **Données relationnelles structurées et métadonnées de l'ERP Odoo :** Schéma relationnel de la base PostgreSQL hébergeant les tables de gestion d'Odoo 17 (res_partner, sale_order, account_move, stock_picking, hr_leave) ainsi que les tables système de sécurité et de structure (ir_model, ir_model_fields, ir_model_access, ir_rule). Ces données ne sont jamais dupliquées dans l'IA ; elles sont consultées à la volée via l'ORM sous le profil de l'utilisateur connecté.

• **Données vectorielles et d'indexation sémantique :** Vecteurs numériques denses de 1024 dimensions générés par le modèle BGE-M3 pour chaque bloc documentaire, indexés au sein d'une base vectorielle confinée en local (ChromaDB), parallèlement à un index inversé de fréquences de termes pour l'algorithme BM25.

Le Tableau 33 synthétise la matrice des flux informationnels entre les différents magasins de données et les modules d'IA.

Tableau 33. Matrice des flux et sources de données exploitées par le système

Source de données	Type de contenu	Mode d'accès	Rôle dans le système
Documentation interne (PDF, MD)	Texte non structuré	Indexation vectorielle et lexicale (RAG)	Répondre aux questions procédurales et fonctionnelles
Base relationnelle Odoo (PostgreSQL)	Données métier transactionnelles	ORM Odoo temps réel sous uid strict	Fournir l'état actuel des ventes, stocks et factures
Métadonnées Odoo (ir.model, ir.rule)	Définition du schéma et règles RBAC	Inspection de métadonnées sans sudo()	Valider la visibilité et guider la construction des requêtes
Base vectorielle locale (ChromaDB)	Embeddings 1024d + métadonnées	Recherche par similarité cosinus (top-k)	Sélection des passages les plus proches de la requête
Journal d'audit (miro.ai.log)	Logs des requêtes, durées et sources	Écriture asynchrone par le contrôleur	Assurer la conformité, la traçabilité et l'auditabilité

6.3. Choix du modèle IA et justification

Dans une architecture d'IA souveraine opérant sur une infrastructure privée (instance AWS dédiée en déploiement, hôte Kaggle temporaire pendant le développement), le choix des modèles d'embedding et des modèles de fondation génératifs obéit à un compromis strict entre qualité du raisonnement, fidélité aux sources, latence d'inférence et contrainte de mémoire VRAM.

Modèle d'embedding sémantique retenu : BGE-M3 (BAAI)

Le modèle BGE-M3 (BAAI/bge-m3) a été sélectionné pour la représentation vectorielle des connaissances d'entreprise. Il présente des avantages déterminants pour notre cas d'usage :

- **Polyvalence multi-représentation :** BGE-M3 est capable de générer simultanément des embeddings denses (captant la sémantique globale), des poids lexicaux clairsemés (assimilables au BM25 pour les mots-clés rares) et des représentations multi-vecteurs (ColBERT) pour un alignement précis token à token.
- **Support multilingue de premier plan :** Entraîné sur plus de 100 langues, il offre une excellente compréhension du français professionnel, des terminologies juridico-comptables et des acronymes techniques propres à Odoo.
- **Fenêtre de contexte étendue :** Capable de traiter des séquences atteignant 8 192 tokens, évitant la troncation brutale des sections de manuels techniques complexes.

Modèles de langage pour la génération locale (LLM)

Pour la synthèse des réponses et le raisonnement agentique, plusieurs modèles ouverts de nouvelle génération ont été évalués sur l'infrastructure matérielle locale de Miro K.E. Le Tableau 34 présente l'analyse comparative des modèles candidats.

Tableau 34. Analyse comparative des modèles de langage candidats pour l'inférence locale

Modèle candidat	Paramètres	VRAM requise (quantifié 4/8-bit)	Fidélité au contexte (Anti-hallucination)	Qualité du français	Verdict et décision
Qwen3 (14B)	14 milliards	9 à 11 Go	Excellente (suivi strict des instructions)	Très élevée	Retenu : Modèle principal pour les requêtes complexes et l'ORM
Llama 3.1 (8B)	8 milliards	5 à 6 Go	Très bonne	Bonne	Alternative légère, non utilisée dans la configuration actuelle

Mistral (7B v0.3)	7 milliards	5 à 6 Go	Moyenne (tendance à la prolixité)	Excellente	Alternative de repli pour la recherche documentaire
Phi-3.5 Mini (3.8B)	3.8 milliards	3 Go	Moyenne sur raisonnements ERP complexes	Moyenne	Écarté : Capacités de raisonnement multi-tables insuffisantes

Le modèle Qwen3 (14B) quantifié en format GGUF Q4_K_M est retenu comme moteur génératif de référence. Il combine une bonne capacité de raisonnement et de suivi des instructions et un traitement correct de la langue française ; sur le serveur d'inférence, il est entièrement chargé en mémoire GPU avec une fenêtre de contexte de 32 768 unités. Pour les environnements plus contraints, Llama 3.1 (8B) reste une alternative envisagée, non utilisée dans la configuration actuelle.

Tableau 35. Paramétrage des hyperparamètres d'indexation et d'inférence

Paramètre	Valeur retenue	Justification technique
Taille des chunks (Chunk size)	512 unités	Granularité optimale pour isoler une règle métier sans bruit contextuel
Recouvrement (Chunk overlap)	50 unités	Préservation de la continuité syntaxique entre paragraphes adjacents
Top-k documents récupérés	5 passages	Couverture suffisante pour la synthèse sans saturer la fenêtre de contexte
Pondération RRF (Dense vs BM25)	k = 60 ; poids 0,6 (dense) et 0,3 (BM25)	Fusion robuste équilibrant sémantique et correspondance lexicale exacte
Température de génération (LLM)	0,3 (réponses documentaires)	Génération peu variable pour limiter l'extrapolation ; la température par défaut du client LLM est nulle

6.4. Méthode d’intégration logicielle

L'intégration entre l'ERP Odoo 17 et la couche d'intelligence artificielle repose sur un modèle d'architecture en microservice découplé, interconnecté par des API REST sécurisées et asynchrones. Ce choix architectural répond à des impératifs critiques :

• **Isolation des ressources de calcul :** L'inférence des grands modèles de langage sollicite intensivement la mémoire GPU et le processeur. Découpler le moteur IA sous un service FastAPI distinct empêche tout risque de saturation des workers HTTP standards d'Odoo, garantissant la fluidité ininterrompue des opérations de gestion quotidiennes.

• **Sécurité et authenticité des flux (HMAC SHA-256) :** Chaque requête transmise par le contrôleur Odoo au backend IA est signée cryptographiquement à l'aide d'un secret partagé (signing_secret) configuré dans les paramètres de miro_ai. Une clé de requête éphémère (requester_key) à usage unique est générée par job, interdisant toute attaque par rejeu ou accès externe non authentifié.

• **Pattern de traitement asynchrone (Job Polling) :** En raison du délai de traitement, de quelques secondes à plus d'une minute selon la complexité de la question (voir la section 9.3), la requête est traitée de manière non bloquante. Le contrôleur Odoo soumet la tâche et reçoit immédiatement un identifiant unique (job_id) avec le code HTTP 202 Accepted. Le composant OWL de l'interface interroge ensuite l'état du job toutes les 2 secondes jusqu'à la mise à disposition de la réponse complète.

• **Maîtrise de l'infrastructure d'inférence :** Aucun fournisseur d'IA externe n'est appelé et il n'existe aucun repli implicite vers un service tiers : l'adresse du serveur d'inférence est fournie par l'environnement et un hôte distant doit être explicitement autorisé. Le serveur d'inférence est une instance AWS privée en déploiement ; un hôte Kaggle exposé par un tunnel ngrok sert de cible temporaire pour les essais.

Le Tableau 36 détaille le contrat d'interface des points d'accès exposés entre le module Odoo miro_ai et le middleware FastAPI.

Tableau 36. Spécification des points d'accès API (Endpoints) entre miro_ai et le backend IA

Méthode & Route	Émetteur -> Récepteur	Données transmises	Réponse et fonction opérationnelle
GET /api/v1/health	Odoo -> FastAPI	Jeton HMAC (compte administrateur)	HTTP 200 {status: ok} : Vérification de la disponibilité du serveur IA
POST /api/v1/ask/jobs	Odoo -> FastAPI	question, uid, company_id, groups, session_id, HMAC	HTTP 202 {job_id, status: pending} : Lancement asynchrone de l'analyse
GET /api/v1/ask/jobs/{id}	Odoo -> FastAPI	job_id, headers HMAC	HTTP 200 {status: done, answer, sources, confidence} : Restitution du résultat
POST /miro_ai/sessions	Web Client OWL -> Odoo	limit	HTTP 200 : liste des conversations récentes de l'utilisateur, lue dans le journal miro.ai.log

POST /miro_ai/ask	Web Client OWL -> Odoo	question, session_id, source_mode	Relais contrôleur Odoo : initie le job et retourne le job_id au widget
POST /miro_ai/ask_result	Web Client OWL -> Odoo	job_id, session_id	Scrutation périodique : retourne pending ou la réponse formatée avec sources

PARTIE III : RÉALISATION

Après avoir mené l'analyse préalable des besoins et formalisé la conception conceptuelle et technique des composants d'intelligence artificielle, cette troisième partie est consacrée à la réalisation du système au sein de l'écosystème Odoo 17. Elle s'articule autour de quatre chapitres : l'environnement de mise en œuvre (Chapitre 7), le développement du module Odoo miro_ai et des moteurs documentaire et ORM (Chapitre 8), les tests et l'évaluation des résultats (Chapitre 9) et, en perspective, l'approche Code Intelligence, dont la réalisation reste partielle (Chapitre 10).

Chapitre 7. Environnement de mise en œuvre

Ce chapitre décrit l'environnement technique dans lequel MIRO AI a été développé et exécuté, l'organisation du dépôt de code et de sa configuration, ainsi que la procédure de mise en place sur un nouveau poste. L'objectif est la reproductibilité : le projet ne doit dépendre d'aucun chemin absolu ni d'aucun réglage propre à une machine.

7.1. Environnement technique

Le Tableau 37 recense les composants utilisés. L'ensemble tourne sous Linux (Debian) dans un environnement virtuel Python. L'inférence est confiée à un serveur compatible avec l'API OpenAI (Ollama ou vLLM) dont l'adresse est fournie par l'environnement ; aucun fournisseur d'IA externe n'est appelé et il n'existe aucun repli implicite vers un service tiers.

Deux cibles d'inférence sont prévues, choisies par la variable LLM_TARGET. La cible aws, par défaut, est une instance AWS privée qui sert de serveur d'inférence de déploiement. La cible kaggle est un hôte de test temporaire : un notebook Kaggle exposé par un tunnel ngrok, utilisé pour les essais pendant le développement. Cette seconde cible est isolée dans une seule entrée de la configuration et se retire en supprimant cette entrée et les variables KAGGLE_*. Le serveur contrôlé le 26 octobre 2026 exécute Ollama 0.40.8 et sert les modèles qwen3:14b (quantifié Q4_K_M, entièrement en mémoire GPU, fenêtre de 32 768 unités) et bge-m3. Un hôte distant doit être explicitement autorisé : par défaut, seules les adresses locales le sont.

Tableau 37. Composants de l'environnement de développement et d'exécution

Composant	Choix retenu	Rôle dans le système
Système d'exploitation	Debian GNU/Linux	Poste de développement et serveur applicatif
Langage	Python 3.11 en environnement virtuel (venv)	Backend IA, ingestion, outils de lecture
ERP	Odoo 17 Community, installé depuis les sources	Instance cible et hébergement du module miro_ai
Base de données	PostgreSQL 14 ou supérieur	Stockage des données d'Odoo
Interface de service	FastAPI exécuté par uvicorn	Middleware d'orchestration et points d'accès
Orchestration	LangGraph	Classification, routage et vérification des réponses
Serveur d'inférence	Ollama, compatible vLLM ; instance AWS (déploiement) ou hôte Kaggle (test)	Génération de texte et calcul des embeddings
Modèle de génération	qwen3:14b, quantifié Q4_K_M	Rédaction des réponses et choix des outils
Modèle d'embedding	bge-m3	Vectorisation des passages documentaires
Base vectorielle	ChromaDB (stockage local persistant)	Index dense des passages
Index lexical	rank_bm25 (BM25Okapi)	Recherche par mots-clés
Gestion de versions	Git, dépôt GitHub	Historique des modifications

7.2. Organisation du dépôt et configuration

Le dépôt regroupe, dans un même ensemble versionné, le module Odoo, le backend et les moteurs de réponse. Chaque répertoire correspond à une responsabilité :

- **api :** points d'accès FastAPI, authentification et traitement des demandes longues
- **core :** graphe LangGraph principal, routage de la question et vérification finale, collecte des preuves
- **document_rag :** ingestion, recherche et génération pour les documents internes
- **erp_data :** outils typés de lecture des données d'Odoo
- **odoo_addons/miro_ai :** module Odoo 17 : interface de conversation et client du backend
- **code_intelligence :** analyse du code source d'Odoo (présentée au Chapitre 10)
- **config, scripts, tests :** constantes de configuration, utilitaires d'installation et d'évaluation, tests automatisés

La configuration obéit à une règle simple. Le fichier d'environnement (.env) ne contient que ce qui change d'un poste ou d'un déploiement à l'autre : secrets, adresses du serveur d'inférence et de

l'instance Odoo, emplacement des installations externes, chacun avec une valeur par défaut portable. Tout le reste (modèles, seuils, limites, comportement) est une constante du fichier config/settings.py. Les dépendances installées sont toutes consignées dans requirements.txt, et chaque modification du projet est inscrite dans un journal de travail horodaté (todo/contexte.txt), ce qui assure la traçabilité des décisions.

7.3. Procédure de mise en place

Le Tableau 38 résume les étapes d'installation sur un nouveau poste, telles que décrites dans le fichier README du projet.

Tableau 38. Étapes de mise en place du système sur un nouveau poste

Étape	Action	Résultat attendu
1. Dépendances et dépôt	Installation des paquets système, clonage du dépôt, création du venv, installation de requirements.txt	Environnement Python reproductible
2. Configuration	Copie de .env.example vers .env : cible d'inférence (LLM_TARGET), adresse du serveur, mot de passe du compte de service Odoo	Secrets conservés hors du dépôt
3. Odoo	Installation des sources 17.0, création de la base PostgreSQL, odoo.conf avec odoo_addons dans addons_path	Instance Odoo démarrable
4. Modules	Installation de miro_ai et des modules métiers listés dans scripts/odoo_modules_to_install.txt	Modules disponibles dans Odoo
5. Données de test	Exécution du script full_setup.py (société, journaux, utilisateurs, produits)	Instance de démonstration
6. Index documentaire	Exécution de document_rag.ingestion.ingest_all	Base ChromaDB et index BM25 construits
7. Démarrage	Lancement d'Odoo puis de uvicorn (api.main:app)	Bouton MIRO AI visible dans la barre supérieure

Chapitre 8. Développement des composants

Ce chapitre présente les composants effectivement développés : le module Odoo miro_ai, le backend d'orchestration, le moteur de recherche documentaire et les outils de lecture des données de l'ERP. Les choix de conception justifiés au Chapitre 6 n'y sont pas répétés ; on y décrit ce que le code réalise.

8.1. Module Odoo miro_ai

Le module miro_ai s'installe comme n'importe quel module Odoo 17. Il se compose des éléments suivants :

• **Contrôleurs :** réception de la question, signature de la requête, transmission au backend et suivi de la tâche (fichiers main.py, ask_flow.py et backend_client.py)

• **Modèles :** paramètres du module (adresse du backend, secret de signature) et journal d'audit des échanges (miro_ai_settings, miro_ai_log)

• **Interface :** composant OWL accessible depuis la barre supérieure, gabarit XML, feuille de style et libellés traduits ; traduction française fournie dans le répertoire i18n

• **Sécurité :** groupes et droits d'accès propres au module

[À insérer : capture d'écran du bouton MIRO AI dans la barre supérieure d'Odoo et du panneau de conversation ouvert]

Figure 8.1 — Interface de conversation du module miro_ai dans Odoo 17

8.2. Backend d'orchestration

Le backend FastAPI expose les points d'accès décrits au Tableau 36. La demande de réponse est asynchrone : le point d'accès de création de tâche renvoie immédiatement un identifiant avec le code 202, puis un second point d'accès permet de consulter l'état de la tâche jusqu'à la réponse complète. Des points d'accès complémentaires permettent de relancer l'indexation documentaire, de consulter son état, de déclencher un balayage planifié, de contrôler la santé du service et de lister les modules couverts.

Pour chaque question, le graphe LangGraph choisit d'abord la source : le choix explicite de l'utilisateur est prioritaire ; à défaut, une classification détermine la source la plus probable, et le système demande une précision lorsque la confiance est insuffisante au lieu de choisir silencieusement. La question suit ensuite l'un des deux chemins (documentation ou système Odoo), les observations collectées sont rassemblées dans un dossier de preuves, puis une vérification finale précède la réponse.

8.3. Moteur de recherche documentaire

8.3.1. Ingestion des documents

L'ingestion lit les fichiers PDF, DOCX, Markdown et texte. Les PDF numérisés peuvent être traités par reconnaissance optique de caractères (pytesseract et pdf2image). Chaque document est découpé en passages de 512 unités avec un recouvrement de 50, et chaque passage reçoit des métadonnées (position dans le document, empreinte du contenu). L'empreinte permet une ingestion incrémentale : seuls les fichiers modifiés sont réindexés. Un observateur de dossier réagit aux ajouts et modifications, et un balayage périodique (toutes les six heures) rattrape les changements manqués.

Le corpus de démonstration compte dix-huit documents (six PDF, deux DOCX, trois fichiers Markdown et sept fichiers texte) couvrant les achats, la facturation, la clôture comptable, les notes de frais, les ressources humaines, la sécurité informatique et le support. En production, le

corpus est destiné à vivre dans un dossier Google Drive partagé : un compte de service en lecture seule lit les dossiers approuvés, un corpus approuvé correspond à une collection, et la synchronisation supprime les passages dont le document n'est plus dans le périmètre.

8.3.2. Recherche hybride

La recherche combine une recherche vectorielle dans ChromaDB (embeddings bge-m3) et une recherche lexicale BM25 avec normalisation Unicode et suppression des mots vides français et anglais. Les deux classements sont fusionnés par Reciprocal Rank Fusion pondérée. Le jeu de candidats est volontairement large avant la sélection finale, les passages contigus des passages retenus sont ajoutés pour restituer le contexte, et les passages issus d'un document Drive qui n'existe plus sont écartés. Un reclasseur de type cross-encoder a été essayé puis retiré, ce qui a permis de supprimer la pile PyTorch des dépendances. Le Tableau 39 donne les paramètres effectifs.

Tableau 39. Paramètres effectifs du moteur de recherche documentaire

Paramètre	Valeur	Remarque
Taille des passages	512	Recouvrement de 50
Modèle d'embedding	bge-m3	Vecteurs denses, collection dédiée en 1024 dimensions
Index lexical	BM25Okapi	Mots vides français et anglais
Fusion	Reciprocal Rank Fusion	Poids 0,6 (vectoriel) et 0,3 (lexical)
Passages retournés	5 par défaut	Candidats : 20 fois ce nombre, au moins 100
Extension du contexte	Passages contigus	Ajoutés après la sélection

8.3.3. Génération et vérification

Le modèle de génération reçoit les passages numérotés par des alias courts (S1, S2, ...) propres au tour de conversation. Les consignes lui imposent de citer les sources, de ne déclarer que les alias réellement utilisés, de restituer intégralement les listes, tableaux et étapes présents dans les passages, de signaler les contradictions entre documents et d'indiquer un niveau de confiance. Une vérification déterministe contrôle ensuite que les éléments des listes et tableaux des passages cités figurent bien dans la réponse, et calcule un taux de restitution.

[À insérer : capture d'une réponse documentaire avec ses sources citées et son badge de confiance]

Figure 8.2 — Réponse documentaire avec citation des sources

8.4. Lecture des données de l'ERP

Les questions portant sur des valeurs actuelles (statut, nombre, liste, agrégation) sont traitées par sept outils typés de lecture, présentés au Tableau 40.

Tableau 40. Outils typés de lecture des données d'Odoo

Outil	Fonction
orm_fields_get	Décrire les champs d'un modèle
orm_search	Retrouver des identifiants selon un domaine de recherche
orm_read	Lire des champs d'enregistrements connus
orm_search_read	Rechercher et lire en une seule opération
orm_search_count	Compter les enregistrements correspondant à un domaine
orm_read_group	Agréger et regrouper (par jour, semaine, mois, trimestre, année)
orm_name_search	Retrouver un enregistrement à partir de son nom

Ces outils offrent plusieurs garanties. Aucune opération de création, de modification ou de suppression n'est exposée. Les entrées (nom de modèle, domaine, champs, tri, regroupement) sont validées par des expressions régulières et une liste fermée d'opérateurs. Le nombre de lignes retournées est plafonné par une constante de configuration. L'exécution se fait sous l'identifiant de l'utilisateur authentifié et dans le contexte de sa société, sans élévation de privilèges : l'identité ne provient jamais du modèle de langage. Chaque résultat est renvoyé sous forme structurée et horodatée.

Chapitre 9. Tests et évaluation

Ce chapitre rend compte des vérifications effectuées. Il distingue les tests logiciels, qui établissent que le code fait ce qui est prévu, de l'évaluation de la qualité des réponses, qui ne s'en déduit pas. Un test logiciel réussi ne prouve pas la justesse métier des réponses sur toutes les instances Odoo 17.

9.1. Tests automatisés

Les tests, écrits avec pytest, couvrent le routage, la résolution des termes, les outils, la construction du graphe, l'ingestion et la recherche documentaires, et les outils de lecture ORM. Les tests d'intégration exigent une instance Odoo démarrée et des utilisateurs de test ; ils sont séparés des tests unitaires par un marqueur. Lors de la dernière exécution complète, le 27 octobre 2026, la suite a donné 1 036 tests réussis, 17 ignorés, 9 échecs et 23 erreurs. Les 23 erreurs proviennent d'utilisateurs de test absents de l'instance (problème d'environnement, non de code). Parmi les 9 échecs, quatre étaient des tests devenus périmés après une simplification du code ; ils ont été corrigés, sans que la suite complète ait été rejouée depuis.

9.2. Banque de questions de référence

Une banque de 37 questions, répartie en quatre catégories (Tableau 41), sert à mesurer la proportion de réponses correctement sourcées et contenant les éléments attendus. L'objectif fixé est d'au moins 90 % de réponses correctes.

Tableau 41. Composition de la banque de questions de référence

Catégorie	Questions	Ce que la catégorie vérifie
Documentaire	10	Retrouver une procédure dans les documents indexés et la restituer avec ses sources
Données ERP chiffrées	15	Obtenir une valeur actuelle par les outils de lecture ORM
Code source	8	Expliquer un comportement établi par le code d'Odoo
Hors périmètre	4	Signaler la limite au lieu d'inventer une réponse
Total	37	

[À compléter : résultat de tests/eval_rag.py (réponses correctes sur 37, par catégorie), à lancer avec Odoo et le serveur d'inférence en marche]

9.3. Séries de conversation

Une série de dix questions métier (droits de validation, états d'un processus, circuit d'approbation d'un achat, question sur un sujet absent des documents) a été rejouée de bout en bout le 14 octobre 2026 avec deux profils : un administrateur et un employé ne disposant pas des droits de validation. Le second profil sert à distinguer un refus légitime d'un refus erroné. Les dix questions ont obtenu une réponse sans tentative invalidée, neuf par le chemin d'analyse du système et une par le chemin documentaire.

La durée de traitement a varié de 44 à 116 secondes par question, soit une moyenne d'environ 90 secondes. Ce délai est très supérieur à celui d'un appel d'inférence simple évoqué à la section 6.4, car le chemin d'analyse enchaîne plusieurs appels d'outils et de modèle. Il constitue une limite d'usage actuelle.

9.4. Budget de contexte

Le modèle de génération dispose d'une fenêtre de 32 768 unités. Sur trois questions de test (achats, stock, ventes), les observations injectées ont occupé respectivement 38,4 %, 39,4 % et 21,1 % de cette fenêtre, sans qu'aucune coupe ni réduction ne soit nécessaire.

9.5. Limites de l'évaluation

- chaque question a été rejouée une seule fois, alors que les réponses du modèle ne sont pas déterministes
- la latence du chemin documentaire n'a pas fait l'objet d'une mesure séparée

• les tests d'intégration dépendent d'une instance Odoo et d'utilisateurs de test qui ne sont pas fournis avec le dépôt

• la justesse métier n'a été relue manuellement que sur un échantillon de questions (voir Chapitre 10)

Chapitre 10. Perspectives : l'agent Code Intelligence

Le temps imparti n'a pas permis de mener l'agent Code Intelligence au niveau de maturité d'une réalisation livrée. Il est présenté ici comme une perspective : on expose son approche, ce qui a été validé et ce qui reste à faire.

10.1. Problème et objectif

Un assistant fondé sur un modèle de langage peut formuler une réponse plausible sans qu'aucune source ne l'étaie, par exemple affirmer qu'un module est absent d'une instance parce qu'une recherche n'a rien retourné. Pour un utilisateur métier, une telle réponse est plus dangereuse qu'un aveu d'ignorance. Code Intelligence vise à répondre aux questions sur le fonctionnement réel d'une instance Odoo en s'appuyant sur le code, la configuration et, lorsque la question l'exige, les données de l'instance, et à signaler explicitement les limites de ce qu'il a pu établir.

10.2. Approche

L'approche repose sur quatre éléments. Un extracteur lit le code Python, les vues XML et les fichiers de droits des modules installés. Un graphe de connaissances relie modèles, champs, vues, boutons, menus, groupes, règles d'accès, états et méthodes. Des analyses déterministes (sans modèle de langage) calculent ce que le code établit : droits d'accès, chemins de navigation, états et transitions d'un processus, chaîne de surcharge d'une méthode, verrous de validation. Enfin un agent unique, qui ne dispose que d'un ensemble borné d'outils, enchaîne ces analyses et rédige l'explication, chaque affirmation devant s'appuyer sur une observation collectée.

Sur l'instance de développement, le graphe actif compte 25 235 nœuds et 25 670 arêtes. Il couvre 288 des 289 modules déclarés installés, soit 17 452 fichiers source indexés ; le module dont le code source est absent est signalé comme écart de couverture. Les outils exposés à l'agent sont regroupés par famille de questions dans le Tableau 42.

Tableau 42. Familles de questions et outils de Code Intelligence

Famille de questions	Outils exposés à l'agent
Découverte et lecture du code	search_system, read_source_fragment
Droits et visibilité	who_has_access_to, evaluate_record_rules, describe_navigation_path, button_execution_path, analyze_security_locks
Approbations et processus	describe_workflow, analyze_method_override_chain, analyze_server_actions

Famille de questions	Outils exposés à l'agent
Saisie et validations	analyze_data_constraints
Données actuelles	Outils de lecture ORM du Tableau 40

10.3. Principes de fiabilité

- **Preuve avant affirmation :** un résultat de recherche sert à découvrir ; une affirmation s'appuie sur une inspection ou une lecture du code
- **Code et exécution distingués :** le code montre les mécanismes déclarés ; seules les données de l'instance montrent l'effet pour un utilisateur donné
- **Prudence sur les cibles :** un terme métier ambigu déclenche une demande de précision plutôt qu'un choix arbitraire
- **Honnêteté sur la couverture :** une recherche vide n'est pas une preuve d'absence lorsque la couverture est incomplète
- **Respect des accès :** le modèle de langage ne choisit ni l'utilisateur ni la société, et les lectures restent en lecture seule

10.4. Illustration

À la question « demande d'achat », l'agent a décrit le processus de validation d'une demande dans un module personnalisé : seul un membre du groupe des gestionnaires d'achats ou un utilisateur d'une liste d'exceptions peut valider ; au-delà de 10 000 euros, seuls les utilisateurs de la liste restreinte le peuvent ; sinon un message d'erreur est affiché. Ces règles proviennent de la méthode de validation du module, dont la réponse cite le fichier et les lignes, ainsi que la chaîne de surcharge correspondante. L'état de la demande (brouillon, à valider, validé, refusé) et les deux méthodes qui l'écrivent sont également restitués.

[À insérer : capture de la réponse de MIRO à « demande d'achat » avec la section Sources]

Figure 10.1 — Réponse de Code Intelligence sur la validation d'une demande d'achat

10.5. État d'avancement

Tableau 43. Bilan de Code Intelligence à la date de rédaction

Élément	État
Construction du graphe sur les modules installés	Validée : couverture de 288 modules sur 289 déclarés
États et méthodes qui écrivent l'état d'un processus	Validés, correction apportée le 27 octobre 2026
Chaîne de surcharge et verrous de validation	Validés sur des cas réels, avec citation du fichier et des lignes

Élément	État
Chemin de navigation, champs et boutons d'un modèle	Correction codée le 27 octobre 2026, à rejouer sur l'instance
Résolution d'un terme métier vers le bon modèle	Partielle : 31 cibles exactes sur 48 questions (audit du 18 octobre 2026)
Justesse des réponses relues le 22 octobre 2026 (achats, ventes, stock)	Contrastée : trois réponses exactes, six partielles, trois fausses, une sans réponse, les autres mêlant bonne conclusion et justification contaminée
Refus en l'absence de preuve	Insuffisant : une question sur les articles achetés a reçu une réponse sans source affirmant que le module Achats est absent

10.6. Suite envisagée

- transformer l'absence de preuve en refus ou en demande de précision, au lieu d'une réponse rédigée de mémoire
- relier les questions de données aux outils de lecture ORM, sous l'identité du demandeur
- améliorer la résolution des termes métier vers les modèles Odoo, mesurée sur une banque de questions à vérité établie
- rejouer un protocole d'évaluation figé sur des instances et des cas de test définis avant tout objectif chiffré de précision
- étendre l'approche aux versions plus anciennes d'Odoo rencontrées chez d'autres clients, qui exigeront de nouveaux extracteurs

CONCLUSION

Ce mémoire a porté sur la conception et la réalisation d'un assistant conversationnel privé intégré à Odoo 17, au sein de l'entreprise MIRO K.E. L'objectif était de permettre aux utilisateurs métier d'obtenir des réponses compréhensibles, sourcées et conformes à leurs droits d'accès, sans confier les données de l'entreprise à un fournisseur d'intelligence artificielle externe.

Le système réalisé repose sur deux composants. Le module Odoo miro_ai fournit l'interface de conversation dans la barre supérieure d'Odoo, signe chaque requête par HMAC, relaie l'identité de l'utilisateur et sa société, suit la tâche de façon asynchrone et journalise les échanges. Le backend FastAPI, orchestré par LangGraph, choisit la source de la réponse : un moteur de recherche documentaire hybride, qui fusionne une recherche vectorielle (bge-m3) et une recherche lexicale (BM25) par Reciprocal Rank Fusion et cite ses sources, ou des outils typés

de lecture des données d'Odoo, en lecture seule et sous l'identité de l'utilisateur. La génération est confiée à un modèle ouvert, Qwen3 14B, servi par un serveur d'inférence privé : une instance AWS en déploiement et un hôte Kaggle temporaire pour les essais.

Les vérifications effectuées montrent un système fonctionnel de bout en bout : la suite de tests automatisés compte plus d'un millier de tests réussis, et une série de dix questions métier a obtenu une réponse avec deux profils d'utilisateurs. Elles révèlent aussi des limites que nous assumons : un temps de réponse de l'ordre de la minute sur le chemin d'analyse du système, une évaluation de la justesse métier encore partielle et des tests d'intégration qui dépendent d'une instance Odoo préparée. Un test logiciel réussi ne prouve pas à lui seul la précision métier des réponses ; celle-ci doit être confirmée par une évaluation reproductible sur des cas définis.

Plusieurs perspectives prolongent ce travail. La principale est l'agent Code Intelligence, présenté au Chapitre 10 : en construisant un graphe du code et de la configuration de l'instance, il vise à fonder chaque réponse sur des preuves consultées et à signaler ce qu'il ne peut pas établir. Il a été validé sur des cas réels de validation, d'états de processus et de surcharge de méthodes, mais sa résolution des termes métier et son refus en l'absence de preuve restent à fiabiliser. S'y ajoutent le serveur MCP dédié à Odoo évoqué au Chapitre 3, l'extension aux versions plus anciennes d'Odoo rencontrées chez d'autres clients et la rédaction d'un protocole d'évaluation figé.

Sur le plan personnel, ce travail m'a permis de mettre en pratique, sur un cas concret, les connaissances acquises à l'École Nationale d'Informatique, et d'approfondir le développement de modules Odoo, la construction d'un service d'intelligence artificielle privé et la rigueur qui consiste à distinguer ce qu'un système établit de ce qu'il suppose.

BIBLIOGRAPHIE

WEBOGRAPHIE

[6]. https://www.python.org/downloads/source/, Téléchargement de python, consulté le 03 Septembre 2024 ;

[7]. https://code.visualstudio.com/download, Téléchargement de Visual studio Code, consulté le 04 Septembre 2024 ;

[8]. https://nightly.odoo.com/17.0/nightly/src/, Téléchargement d'Odoo 17 Communautaire, consulté le 05 Septembre 2024 ;

[9].https://www.odoo.com/documentation/17.0/fr/administration/on_premise/source.html, Documentation pour l'installation d'Odoo 17 Communautaire, consulté le 05 Septembre 2024 ;

[10]. https://www.postgresql.org/download/linux/, Documentation pour le téléchargement et l'installation de PostgreSQL, consulté le 07 Septembre 2024 ;

[11]. https://apps.odoo.com/apps/modules/browse?search=apps+menu, dans le cadre de l'exploration de l'idée de vendre le module « Apps Menu », consulté le 10 septembre 2024 ;

[12]. https://apps.odoo.com/apps/modules/browse?search=brevo+connector, dans le cadre de l'exploration de l'idée de vendre le module « Brevo connector », consulté le 15 septembre 2024 ;

[13]. https://pypi.org/project/sib-api-v3-sdk/, lien pour le téléchargement et l'installation de sib-api-v3-sdk consulté le 28 Septembre 2024 ;

[14].https://www.cybrosys.com/odoo/odoo-books/v17/marketing-automation/, Documentation de la fonctionnalité d'automatisation du marketing dans Odoo 17 entreprise, consultée le 28 septembre 2024 ;

[15].https://developers.brevo.com/reference/getting-started-1, Documentation pour l'intégration de l'api campagne d'emailing Brevo consulté le 28 Septembre 2024 ;

[16].https://openclassrooms.com/fr/courses?query=MVC, Documentation de l'architecture MVC , consulté le 30 Septembre 2024 ;

[17].https://www.apik.cloud/historique-odoo, Historique d'Odoo, consulté le 20 novembre.

[18] https://www.data-bird.co/blog/langage-python, Langage Python : Qu'est ce que le langage Python ? consulté le 21 Novembre 2024 ;

[19] https://thecodest.co/fr/blog/avantages-et-inconvenients-de-python/, Avantages et inconvénients du Python, consulté le 23 Novembre 2024 ;
[20] https://hightech.savoir.fr/cours-merise-pdf/ , Cours merise pdf : merise avantages et inconvénients**,** consulté le 25 Novembre 2024 ;
[21].https://fr.scribd.com/document/563561536/2tupLesson, Avantages et inconvénients de la méthode 2TUP, consulté le 26 Novembre 2024 ;
[22] https://innowise.com/fr/blog/benefits-and-drawbacks-of-java/, Avantages et inconvénients du langage Java, consulté le 26 Novembre 2024 ;
[23]. https://www.ambient-it.net/avantages-inconvenients-reactjs/, Avantages et inconvénients du Framework React js, consulté le 28 Novembre 2024 ;
[24]. https://www.odoo.com/fr_FR/blog/actualites-d-odoo-5/introducing-owl-the-fastest-javascript-framework-in-the-world-1047, Avantages et inconvénients du Framework Owl js, consulté le 28 Novembre 2024 ;
[25]. https://www.oracle.com/fr/database/postgresql-versus-mysql/, Comparaison de MySQL et PostgreSQL ,consulté le 29 Novembre 2024 ;
[26]. https://www.oracle.com/fr/erp/what-is-erp/, Qu’est ce qu’un système ERP ?, consulté le 30 Novembre 2024 ;

GLOSSAIRE

Brevo : Plateforme d'emailing et de l'automatisation du marketing, anciennement connue sous le nom de Sendinblue.

Campagne d'emailing : Envoi massif d'emails à un groupe cible dans le but de promouvoir un produit, service ou message.

Contact : une information d'une personne ou une organisation avec laquelle l'entreprise interagit via des actions de marketing et de gestion des emails.

Clics uniques : Nombre de clics uniques sur les liens dans l'email.

Différé : Nombre d'emails différés (temporaires), souvent à cause de problèmes avec le serveur du destinataire**.**

Email : Message électronique envoyé d'un expéditeur à un ou plusieurs destinataires via Internet.

Email marketing : Utilisation de l'email comme canal de communication pour des campagnes publicitaires ou informatives.

Étiquette : Mot-clé ou label utilisé pour classer, organiser ou identifier des éléments.

Expéditeur : Personne ou organisation qui envoie un message ou un courrier, identifiable par son nom ou son adresse électronique.

Liste de diffusion : Ensemble d'adresses email regroupées pour permettre l'envoi simultané de messages à tous les membres. Utilisée principalement pour les campagnes de communication ou d'emailing.

Livré : Nombre d'emails livrés avec succès.

Marketing : Ensemble des actions visant à promouvoir des produits ou services pour attirer et fidéliser les clients.

Menu des applications : Interface dans un logiciel qui permet de naviguer entre les différentes applications ou modules disponibles.

Modèle d'email : Template prédéfini utilisé pour créer et personnaliser des emails rapidement.

Audience : Groupe de personnes ciblées pour une campagne de marketing.

Odoo : Logiciel de gestion intégré (ERP) open-source qui permet de gérer différents aspects d'une entreprise, tels que la comptabilité, les ventes et le marketing.

Pièces jointes : Fichiers supplémentaires inclus dans un email ou un document pour fournir des informations ou des ressources complémentaires (par exemple : images, PDF, documents Word, etc.).

Vues uniques : Nombre de vues uniques de l'email**.**

ANNEXE

I. Le logiciel ERP Odoo

1. Historique :

Depuis une petite chambre étudiante à Louvain-la-Neuve jusqu'au statut de licorne valorisée à plus de 5 milliards d'euros, Odoo incarne l'excellence et l'audace. Portée par **Fabien Pinckaers**, son fondateur et visionnaire, l'entreprise s'est imposée comme un acteur majeur des logiciels de gestion d'entreprise, transformant profondément le paysage numérique mondial.

- **En 2004 : TinyERP, une idée révolutionnaire née à Louvain-la-Neuve**

L'histoire commence en 2004, alors que Fabien Pinckaers, étudiant en ingénierie, conçoit TinyERP pour répondre à un problème récurrent des PME : l'accès à des outils de gestion flexibles et abordables. La solution repose sur un modèle open source inédit pour l'époque, permettant aux entreprises de personnaliser leur logiciel en fonction de leurs besoins spécifiques, sans les coûts exorbitants des solutions propriétaires.
Mais les débuts sont difficiles. Avec peu de moyens financiers et une concurrence féroce, TinyERP peine à convaincre. Pourtant, l'enthousiasme de Pinckaers et sa capacité à mobiliser une équipe de développeurs passionnés transforment rapidement le projet en une solution appréciée par un nombre croissant de PME.

- **En 2008 : TinyERP devient OpenERP, un premier tournant stratégique**

Quatre ans après son lancement, TinyERP évolue pour devenir OpenERP, marquant une étape majeure. Ce changement de nom reflète une ambition accrue : aller au-delà des outils de gestion traditionnels et proposer une plateforme modulable capable de répondre aux défis variés des entreprises.
La reconnaissance ne tarde pas. En 2010, Fabien Pinckaers réussit à convaincre des investisseurs prestigieux, notamment Xavier Niel et le fonds Sofinnova, qui injectent 3 millions d'euros. Cet apport financier permet à OpenERP d'accélérer son développement, d'enrichir son offre et de renforcer sa position sur le marché international.

- **En 2014 : Odoo, la naissance d'un écosystème global**

Le véritable tournant survient en 2014, lorsque OpenERP devient Odoo. Ce nouveau nom reflète une transformation stratégique profonde : Odoo n’est plus seulement un ERP, mais une suite complète d’applications professionnelles couvrant des domaines variés tels que la comptabilité, la gestion des stocks, le CRM, le marketing et même la création de sites web.
Le modèle freemium adopté par Odoo démocratise l’accès aux logiciels de gestion. Une version gratuite attire des millions d’utilisateurs, tandis que les options payantes, telles que l’hébergement cloud et les modules avancés, génèrent des revenus croissants. Avec plus de 500 applications natives et une bibliothèque de 30 000 modules développés par une communauté active, Odoo devient l’App Store des entreprises.

2. Évolution des Versions d'Odoo

Odoo 8 : En 2014**,** l'entreprise Odoo a mis à jour la version du logiciel en passant à la V8. Parmi les nouvelles fonctionnalités, on trouve l'introduction de modules pour la gestion de sites web (CMS), enrichissant ainsi l'offre d'applications métier.
Odoo 9 : En 2015, Odoo a mis à jour la version du logiciel en passant à la V9. Cette version a marqué le lancement de Odoo Enterprise, une version payante offrant des fonctionnalités avancées, ainsi qu'une simplification des workflows et une gestion financière améliorée.
Odoo 10 : En 2016, l'entreprise Odoo a mis à jour la version du logiciel en passant à la V10. Parmi les nouvelles fonctionnalités, on trouve l'introduction du module MRP (Manufacturing Resource Planning), permettant une gestion complète de la production, ainsi que des améliorations de l'expérience utilisateur.
Odoo 11 : En 2017, Odoo a mis à jour la version du logiciel en passant à la V11. Cette version a apporté une refonte majeure de l'interface utilisateur pour plus de fluidité, ainsi que le lancement du module de maintenance préventive destiné aux entreprises industrielles.
Odoo 12 : En 2018, Odoo a mis à jour la version du logiciel en passant à la V12. Parmi les nouvelles fonctionnalités, on trouve le support natif pour l'IoT (Internet des Objets) et des améliorations des feuilles de temps et des panneaux Kanban interactifs.
Odoo 13 : En 2019, Odoo a mis à jour la version du logiciel en passant à la V13. Cette version a ajouté les applications eLearning et Marketing Automation, tout en optimisant les performances pour les grandes bases de données.

Odoo 14 : En 2020, Odoo a mis à jour la version du logiciel en passant à la V14. Parmi les nouvelles fonctionnalités, on trouve des widgets avancés pour les formulaires, des nouveaux tableaux de bord analytiques et des améliorations des feuilles de calcul intégrées.
Odoo 15 : En 2021, Odoo a mis à jour la version du logiciel en passant à la V15. Cette version a simplifié les flux de travail pour les utilisateurs finaux.
Odoo 16 : En 2022, Odoo a mis à jour la version du logiciel en passant à la V16. Parmi les nouvelles fonctionnalités, on trouve une refactorisation des performances pour accélérer les traitements, ainsi que des améliorations de la gestion documentaire et des applications comptables.
Odoo 17 : En 2023, Odoo a mis à jour la version du logiciel en passant à la V17. Cette version a introduit de nouvelles fonctionnalités collaboratives et des intégrations améliorées avec des services tiers via des API.
Odoo 18 : En octobre 2024, l'entreprise Odoo a mis à jour la version du logiciel en passant à la V18. Parmi les nouvelles fonctionnalités, on trouve l'introduction de filtres personnalisés pour les projets, une démonstration avec des performances inédites (100 produits scannés en 2 secondes), ainsi que l'exportation JSON et une impression optimisée.[17]

TABLE DES MATIERES

CURRICULUM VITAE.....I

SOMMAIRE.....III

REMERCIEMENTS.....V

LISTE DES FIGURES.....VI

LISTE DES TABLEAUX.....IX

LISTE DES ABREVIATIONS.....X

INTRODUCTION GENERALE.....1

PARTIE I : PRESENTATIONS.....2

Chapitre 1. Présentation de l'ENI.....3

1.1. Information d'ordre général.....3

1.2. Missions et historique.....3

1.3. Organigramme institutionnel de l'ENI.....5

1.4. Domaine de spécialisation.....6

1.5. Architecture des formations pédagogiques.....6

1.6. Relations de l'ENI avec les entreprises et les organismes.....9

1.7. Partenariat au niveau international.....10

1.8. Débouchés professionnels et diplômés.....11

1.9. Ressources humaines.....13

Chapitre 2. Présentation de l'établissement d'accueil.....14

2.1. Information de l'entreprise.....14

2.2. Organisation du MIRO KE.....14

2.3. Missions.....14

2.4. Partenariat.....15

2.5. Valeurs fondamentales.....15

Chapitre 3. Description du projet.....17

3.1. Formulation.....17

3.2. Objectif et besoins de l'utilisateur.....17

3.2.1. Objectif.....17

3.2.2. Besoins de l'utilisateur.....17

a) Besoins fonctionnels.....17

b) Besoins techniques.....18
3.3. Moyens nécessaires à la réalisation du projet.....18
3.3.1. Moyens Humains.....18
3.3.2. Moyens Matériels et Logiciel.....19
a) Moyens matériels.....19
b) Moyens logiciels.....19
3.4. Résultats attendus.....20
3.5. Chronogramme de travail.....20
PARTIE II: ANALYSE ET CONCEPTION.....21
Chapitre 4. Analyse préalable.....22
4.1. Analyse de l'existant.....22
4.1.1. Organisation actuelle.....22
4.1.2. Inventaire des moyens matériels et logiciels.....22
a) Matériels existants.....22
b) Logiciel existant.....23
4.2. Critique de l'existant.....23
4.3. Conception avant-projet.....24
4.3.1. Proposition des solutions.....24
4.3.2. Analyse comparative de solutions proposées.....24
4.3.3. Solution retenue.....24
4.3.4. Méthodes de conception et outils utilisés.....25
Chapitre 5. Analyse conceptuelle.....32
5.1. Présentation de la méthode 2TUP.....32
5.2. Dictionnaire des données.....33
5.3. Règles de gestion.....35
5.4. Représentation et spécification des besoins.....36
5.4.1. Diagramme des cas d'utilisation.....37
5.4.2. Description textuelle pour chaque cas d'utilisation.....39
a) Cas d'utilisation : Créer un compte.....39
b) Cas d'utilisation :S'authentifier.....40
c) Cas d'utilisation : Gérer une campagne d'emailing.....40

d) Cas d'utilisation : Gérer modèle d'email.....41
e) Cas d'utilisation : Gérer contacts.....42
f) Cas d'utilisation : Gérer conteneur.....43
g) Cas d'utilisation : Consulter le statistique de campagne d'emailing.....44
h) Planifier l'envois d'emails.....44
5.4.3. Priorisation des cas d'utilisation.....45
5.4.4. Diagramme de séquence système pour chaque cas d'utilisation.....46
5.5. Spécification des besoins techniques.....53
5.6. Modélisation du domaine.....54
Chapitre 6. Conception détaillée.....55
6.1. Architecture système.....55
6.2. Diagramme de séquence de conception pour chaque cas d'utilisation.....56
6.3. Diagramme de classe de conception pour chaque cas d'utilisation.....63
6.4. Diagramme de classe de conception global.....68
6.5. Diagramme de paquetage.....68
6.6. Diagramme de déploiement.....69
PARTIE III : REALISATION.....70
Chapitre 7. Mise en place de l'environnement de développement.....71
7.1. Installation et configuration des outils.....71
7.1.1. Installation de Visual paradigm.....71
7.1.2. Installation de python.....72
7.1.3. Installation de l'IDE Visual Studio Code.....72
7.1.3. Installation de PostgreSQL.....73
7.1.4. Installation et configuration de l'Odoo 17.....74
7.2. Architecture de l'application.....74
Chapitre 8. Développement de l'application.....76
8.1. Création de la base de données.....76
8.2. Codage de l'application.....77
8.3. Présentation de l'application.....80
8.3.1. Présentation des modules.....80
8.3.2. La page d'authentification.....81

8.3.3. Le menu des applications ou page d'accueil.....82
8.3.4. L'application campagne d'emailing brevo.....82
CONCLUSION.....87
WEBOGRAPHIE.....X
GLOSSAIRE.....X
ANNEXE.....XIII
TABLE DES MATIERES.....XVI

RESUMÉ

Ce projet comprend la conception et l'implémentation d'un menu d'application fluide, ainsi que l'intégration des campagnes d'e-mailing Brevo dans Odoo 17 Community pour MIRO K.E. Ces modules visent à améliorer l'expérience utilisateur et à communiquer directement avec les publics cibles. Ce travail a été réalisé dans le cadre de notre formation à l'École Nationale d'Informatique (ENI).

Le développement de ce projet, intitulé « Conception ,réalisation et intégration du module menu des applications et campagne d'emailing brevo dans Odoo 17 », s'est basé sur la méthode 2TUP pour le cycle de développement, le langage de modélisation UML pour la conception, et des technologies modernes telles que OWL js pour le frontend, Python pour le backend, PostgreSQL pour la base de données, et Api brevo pour la communication en temps réel.

Ces modules répondent efficacement aux besoins de l'entreprise pour faciliter l'interaction et l'envoi de campagnes d'emailing.

Mots-clés : Brevo, Email, Campagne, Marketing, Module, Odoo.

ABSTRACT

This project includes the design and implementation of a fluid application menu, as well as the integration of Brevo e-mailing campaigns into Odoo 17 Community for MIRO K.E. These modules aim to enhance the user experience and communicate directly with target audiences. This work was carried out as part of our training at the École Nationale d'Informatique (ENI).

The development of this project, entitled “Design and implementation of the application menu and integration of brevo e-mailing campaigns in Odoo 17 Community”, was based on the 2TUP method for the development cycle, the UML modeling language for the design, and modern technologies such as OWL js for the frontend, Python for the backend, PostgreSQL for the database, and Api brevo for real-time communication.

These modules effectively meet the company's needs to facilitate interaction and the sending of email campaigns.

Keywords: Brevo, Email, Campaign, Marketing, Module, Odoo.